МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по практической работе №3
по дисциплине «Интеллектуальные системы анализа геопространственных данных»
Тема: Сегментация водных объектов: от пороговых методов (OpenCV) до глубокого обучения (U-Net)

Студент гр. 1304 _____ Гирман А.В.

Преподаватель _____ Уткина О.Н.

Санкт-Петербург
2026

Цель работы.

Освоение практических навыков применения методов сегментации изображений для выделения водных объектов на спутниковых снимках, от классических пороговых методов до современных нейросетевых архитектур

Задачи:

1. Освоить классические методы пороговой сегментации (глобальный порог, метод Оцу, адаптивная бинаризация).
2. Изучить метрики оценки качества сегментации (IoU, F1, Precision, Recall).
3. Реализовать архитектуру U-Net для сегментации водных объектов.
4. Освоить технику Transfer Learning с использованием предобученных энкодеров.
5. Научиться настраивать функцию потерь Tversky Loss для управления чувствительностью модели.
6. Провести сравнительный анализ классических и нейросетевых методов.
7. Подготовить данные для последующей векторизации и экспорта в ГИС.

Задание варианта 3.

Регион: Астраханская область

Водный объект: Дельта реки Волги

Координаты (примерные границы): 45.0–46.0° с.ш., 47.0–49.0° в.д.

Теоретический минимум.

Сегментация изображений в экологических задачах

Сегментация — процесс разделения изображения на смысловые области (сегменты). В экологическом мониторинге сегментация позволяет:

- выделить водные объекты (озера, реки, пруды);
- определить зоны загрязнения;
- оценить площадь лесов и вырубок;

- картографировать типы растительности.

а) Глобальный порог

Простейший метод: все пиксели со значением выше порога считаются объектом, ниже — фоном.

Преимущества: простота, быстрота.

Недостатки: не учитывает локальные особенности освещения.

б) Метод Оцу

Автоматически выбирает порог, минимизирующий внутриклассовую дисперсию.

Преимущества: автоматический подбор, оптимален для бимодальных гистограмм.

Недостатки: чувствителен к шуму, работает только при двух четких пиках.

в) Адаптивная бинаризация

Порог вычисляется для каждого пикселя на основе локальной окрестности.

Преимущества: учитывает локальные изменения освещения.

Недостатки: медленнее, требует подбора параметров.

Метрики качества сегментации Intersection over Union (IoU)

Метрика	Формула	Диапазон	Интерпретация
IoU (Intersection over Union)	TP / (TP + FP + FN)	0-1	Отношение пересечения к объединению
F1-score	2 * Precision * Recall / (Precision + Recall)	0-1	Гармоническое среднее
Precision (точность)	TP / (TP + FP)	0-1	Доля правильных положительных

Recall (полнота)	TP / (TP + FN)	0-1	Доля найденных объектов

TP (True Positive) — пиксель правильно определен как вода;

FP (False Positive) — пиксель ошибочно определен как вода;

FN (False Negative) — пиксель воды пропущен;

TN (True Negative) — пиксель правильно определен как не вода

Архитектура U-Net

U-Net — архитектура сверточной нейронной сети, специально разработанная для сегментации биомедицинских изображений, хорошо работает и на спутниковых снимках.

Ключевые особенности:

- skip-connections — пропускные соединения передают пространственные данные от кодирующего блока к декодирующему;
- аугментация данных — увеличивает обучающую выборку;
- функция потерь — обычно Dice Loss или Tversky Loss.

Transfer Learning

Transfer Learning — использование предобученной модели для новой задачи.

Позволяет:

- обучаться на малых датасетах;
- ускорить сходимость;
- достичь более высокой точности

Tversky Loss

Стандартная функция потерь Dice Loss имеет вид:

$$\text{Dice Loss} = 1 - (2*TP + \varepsilon) / (2*TP + FP + FN + \varepsilon)$$

Tversky Loss — обобщение, позволяющее по-разному взвешивать FP и FN:

$$\text{Tversky Loss} = 1 - (TP + \varepsilon) / (TP + \alpha*FN + \beta*FP + \varepsilon)$$

где: α — вес ложноотрицательных ошибок, β — вес ложноположительных ошибок, ε — сглаживание (smoothing).

Экологическая настройка:

Задача	α	β	Обоснование
Обнаружение разливов	0.7	0.3	Пропуск разлива страшнее ложной тревоги
Картографирование водоемов	0.5	0.5	Баланс между ошибками
Мониторинг зарастания	0.3	0.7	Важнее не пропустить зарастание

Выполнение работы.

1. Пороговые методы сегментации.

1.1 В класс *WaterSegmentation* добавлены функции для четырёх методов пороговой сегментации воды. Все методы работают со значением индекса NDWI. В результате работы каждого метода получается бинарная маска с расположением воды.

Индекс рассчитывается методом *_get_ndwi*:

- для 4-канальных данных считается классический NDWI;
- для RGB-изображения, где нет канала NIR, считается его приближение (Green - Blue) / (Green + Blue);

а) Глобальная пороговая сегментация использует один порог для всего изображения. Пиксель относится к воде, если его значение индекса NDWI выше порога *threshold*.

б) Метод Оцу подбирает порог автоматически. Сначала отрицательные значения NDWI приравниваются к нулю, и контраст изображения улучшается выравниванием гистограммы. Затем функция *threshold_otsu* находит порог,

который лучше всего делит гистограмму на два класса. Если порог найти не удалось, в качестве запасного варианта используется медиана NDWI.

в) Адаптивная пороговая сегментация вычисляет свой порог для каждой области изображения. Функция *threshold_local* для каждого пикселя считает средневзвешенное значение NDWI в окне размером *block_size* и вычитает из него смещение *offset*.

г) NDWI-порог специализированный метод для выделения водных объектов. Если во входных данных есть 4 канала, то рассчитывается NDWI, и к воде относятся пиксели со значением выше 0.15. Если канала NIR нет, то метод переходит на изображение в оттенках серого и относит к воде пиксели темнее порога, найденного методом Оцу, так как в видимом диапазоне вода темнее суши.

2. Метрики качества

2.1 Чтобы оценить и сравнить методы пороговой сегментации реализованы функции, которые считают метрики, постобработку (находятся в классе *WaterSegmentation*), тестирование и визуализация (находятся в классе *WaterQualityPipelineExtended*).

Метрики (TP, FP, FN) считаются с помощью полученной маской методом сегментации и эталонной маской воды из датасета. Обе маски переводятся в бинарный вид, где каждый пиксель либо вода, либо суша. Пиксели эталонной маски считаются водой при значении выше 0.5, пиксели предсказанной при значении выше 0.3. Затем пиксели делятся на три группы:

- В TP входят пиксели воды, которые метод правильно отнёс к воде;
- В FP входят пиксели суши, которые метод ошибочно отнёс к воде;
- В FN входят пиксели воды, которые метод пропустил.

По этим значениям рассчитывается основная метрика IoU.

Маски, полученные пороговыми методами, часто содержат отдельные пиксели и мелкие пятна, ошибочно отнесённые к воде, а также небольшие разрывы внутри водоёмов. Для постобработки используется

remove_small_objects, который удаляет связные области меньше *min_size* пикселей и *closing* с круглым элементом радиусом 2 пикселя заполняет мелкие дыры и соединяет близко расположенные участки воды.

Результаты без постобработки показаны на рисунке 1, а с постобработкой на рисунке 2. Итоговые метрики (средний IoU по патчам) можно увидеть в таблице 1.

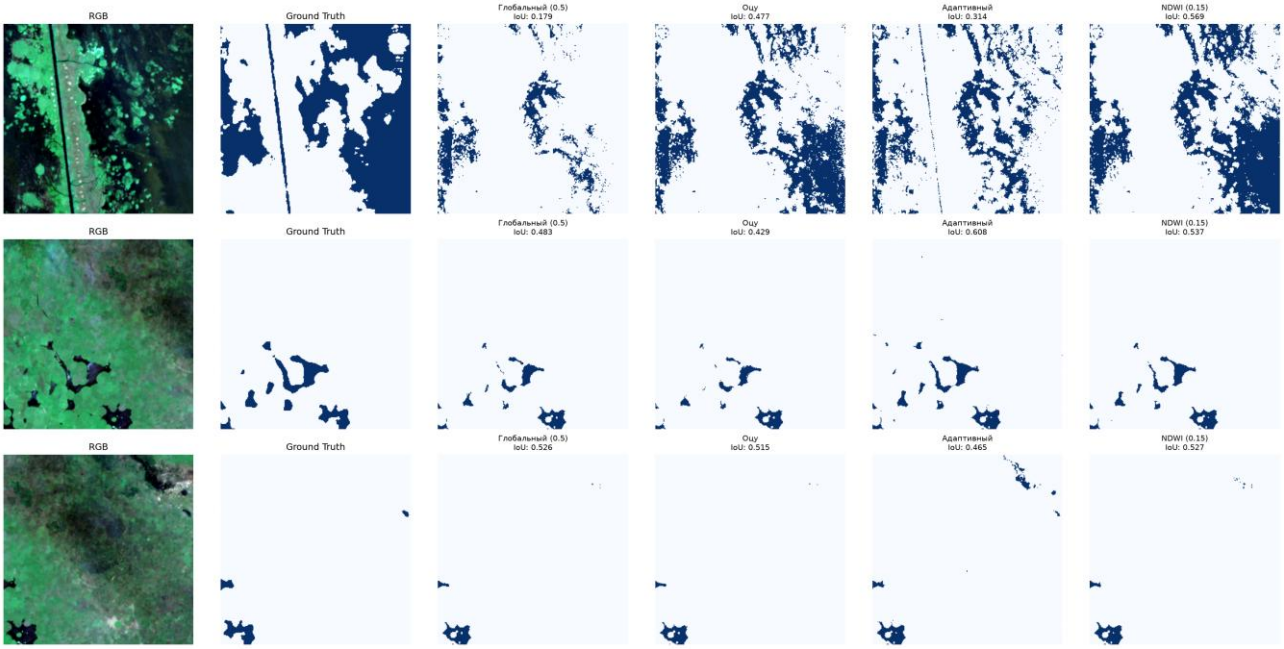


Рисунок 1 – Результаты пороговых методов сегментации без постобработки

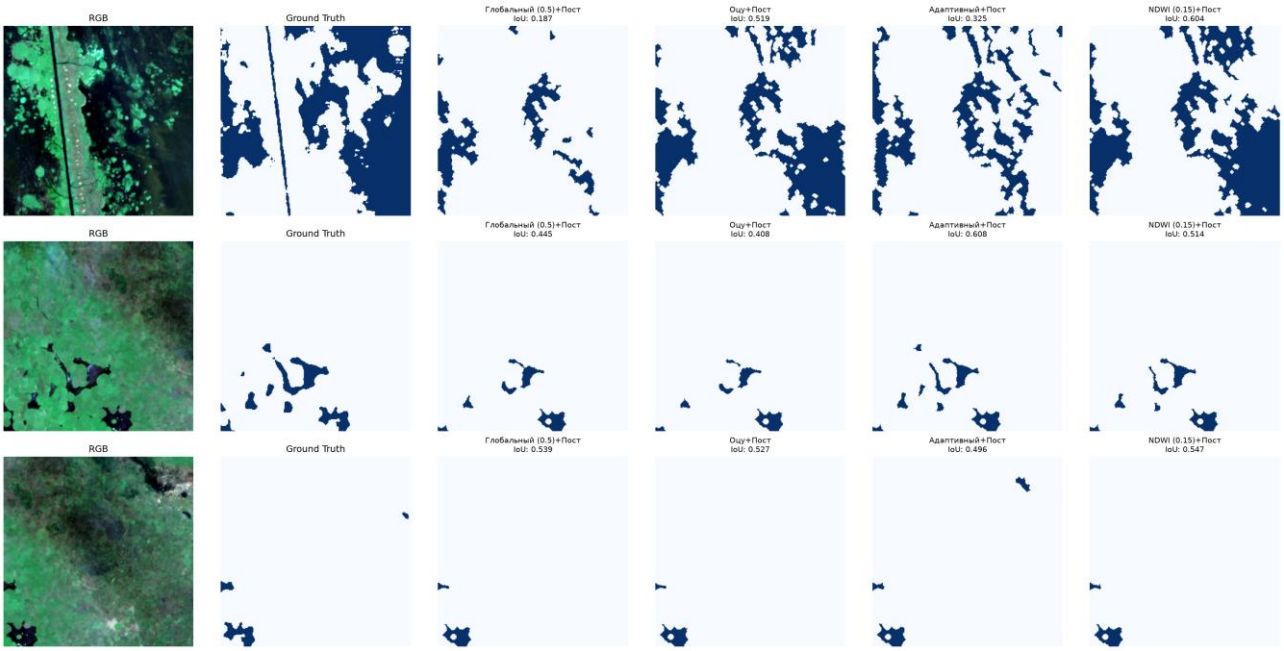


Рисунок 2 – Результаты пороговых методов сегментации с постобработкой

На рисунках 1 и 2 показаны результаты сегментации водных объектов четырьмя методами на трёх фрагментах снимка. Качество оценивалось по метрике IoU относительно эталонной разметки. Без постобработки наилучший средний результат показал порог по NDWI, затем идут метод Оцу и адаптивный порог. Глобальный порог оказался наименее стабильным. На первом фрагменте с большой площадью воды и заболоченными участками он почти не выделил водные объекты, хотя на остальных фрагментах дал более убедительный результат.

Таблица 1 – Итоговые метрики

Метод\Метрика	IoU
Глобальный (0.5)	0.396
Оцу	0.474
Адаптивный	0.462
NDWI (0.15)	0.545
Глобальный (0.5)+Пост	0.390
Оцу+Пост	0.485
Адаптивный+Пост	0.477
NDWI (0.15)+Пост	0.555

В итоге по таблице 1 и анализу рисунков, можно сделать вывод, что наилучшее качество сегментации водных объектов обеспечивает пороговая классификация по индексу NDWI (0.15).

3. Подготовка данных для U-Net

3.1 Для обучения и проверки методов сегментации создан набор патчей с эталонными масками воды. Датасет создаётся из спутникового снимка Sentinel-2 участка дельты Волги. Снимок и эталонная маска загружаются из Google Earth Engine. Выбраны снимки с 1 июня по 1 августа 2025 года. Для выбранного

снимка выгружаются каналы (B4, B3, B2, B8, B11) и карта классов Dynamic World.

Карта *dynamic_world* используется как эталон, потому что она построена моделью, независимой от проверяемых пороговых методов. Патч, маску и наложение можно увидеть на рисунке 3.

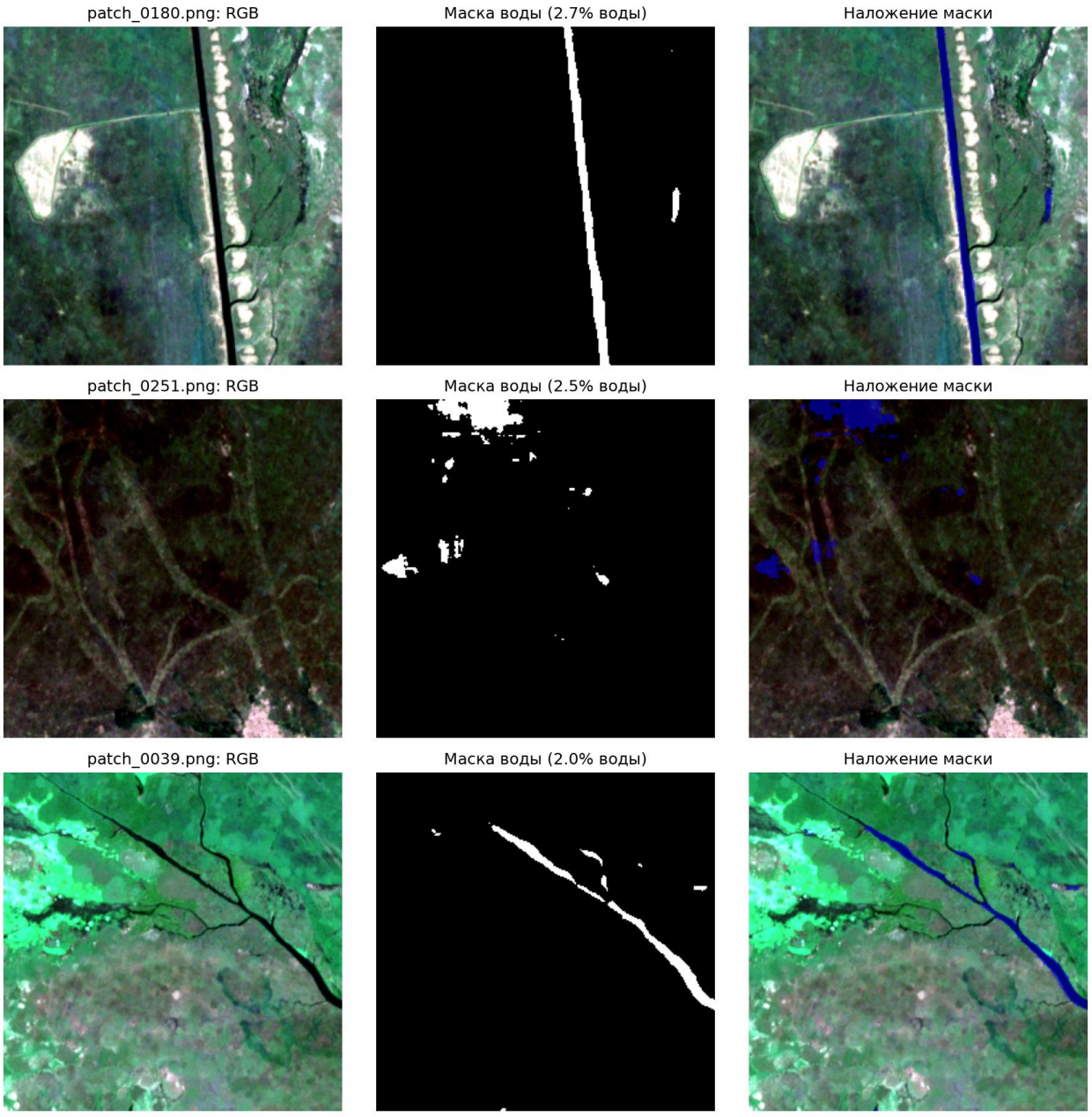


Рисунок 3 – Пример полученного датасета

3.2 Разделение датасета на части выполняет метод *load_dataset* в классе *WaterSegmentation*:

Первая часть (обучающая выборка), на ней модель обучается.

Вторая часть (валидационная выборка), на ней во время обучения проверяется качество и подбираются гиперпараметры.

Третья часть (тестовая выборка), используется для итоговой оценки уже обученной модели.

Для устойчивости обучения выбраны доли 70/15/15 (70% обучающая, 15% валидационная и 15% тестовая выборка). Снимок разделён на клетки 448 на 448 пикселей, и клетки случайно распределены между выборками в заданных долях: 10 клеток отнесены к обучающей выборке, по 2 к валидационной и тестовой. Патч попадает в выборку, только если целиком лежит в клетках этой выборки. Патчи, пересекающие границу клеток разных выборок, исключаются. В итоге получено всего 187 патчей (104 патча исключены), а доля воды у каждого примерно одинакова (см. рис. 4). Такое разделение выбрано потому, что патчи перекрываются. При случайном разбиении по патчам почти одинаковые соседние патчи попали бы и в обучающую, и в тестовую выборку, что привело бы к утечке данных.

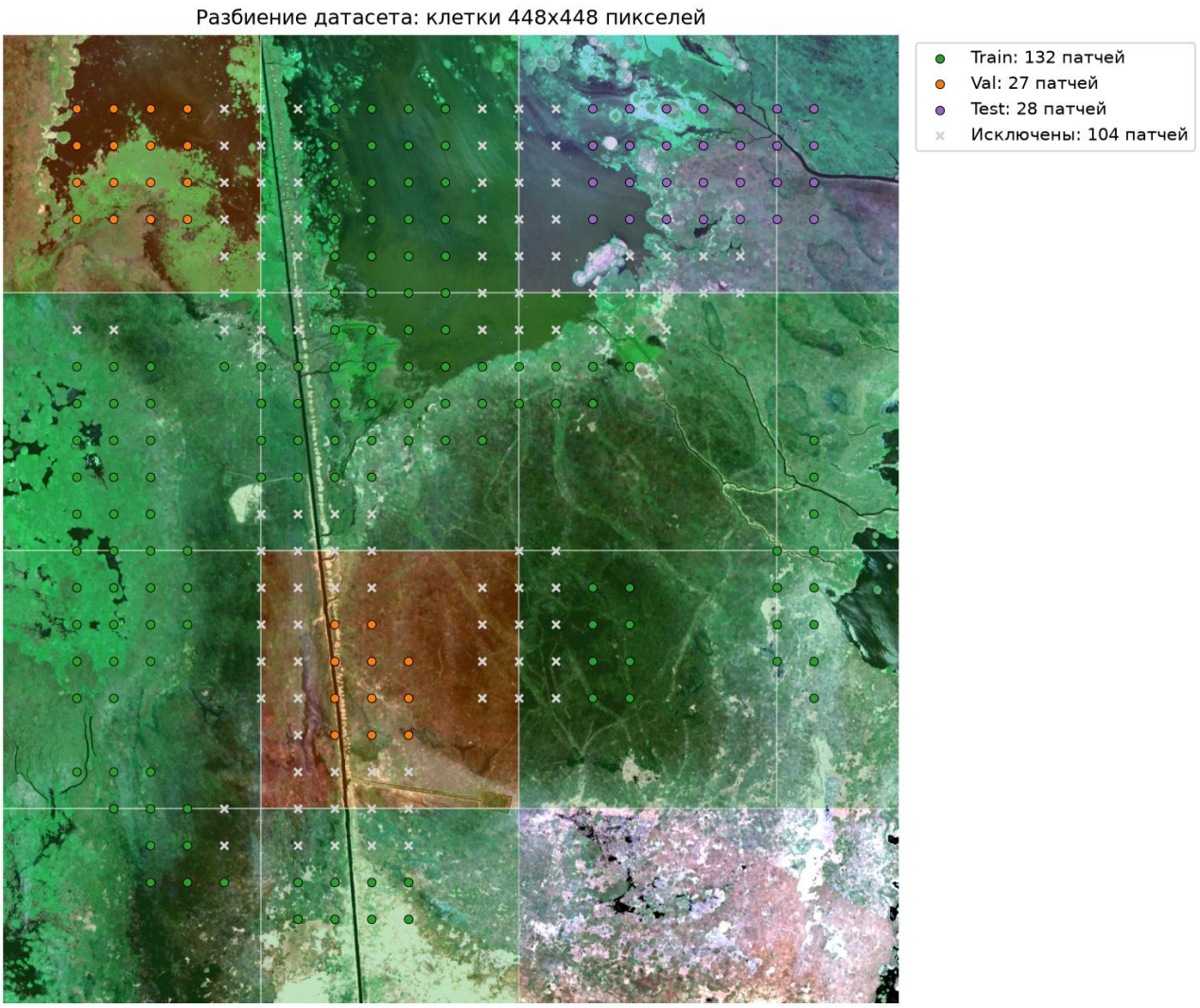


Рисунок 4 – Разбиение датасета

3.3 Для использования аугментации написана функция *get_augmentation*. Она одновременно преобразует изображение и его маску. Геометрические преобразования и сдвиги применяются к обоим одинаково, поэтому маска остаётся совмещённой со снимком. Каждое преобразование срабатывает со своей вероятностью, поэтому каждый раз получается своя случайная комбинация.

Виды преобразований:

- Геометрические преобразования: RandomRotate90 (поворот на градусы кратные 90), HorizontalFlip и VerticalFlip (отражения по горизонтали и вертикали), Transpose (отражение относительно диагонали), Affine (сдвиг, изменение масштаба и поворот);

- Цветовые: RandomBrightnessContrast (яркость и контраст в разбросе 0.2), ColorJitter (яркость, контраст, насыщенность и тон), HueSaturationValue (сдвиг тона, насыщенности и яркости);
- Шумы и размытие: GaussNoise (гауссов шум), ISONoise (шум цифрового сенсора), GaussianBlur и MedianBlur (виды размытия);
- Погодные эффекты: RandomFog (туман), RandomRain (дождь), RandomSnow (снег);
- Вырезания: CoarseDropout (8 чёрных квадратов 32×32);

Примеры использования аугментаций с разной вероятностью, можно увидеть на рисунках 5 и 6.

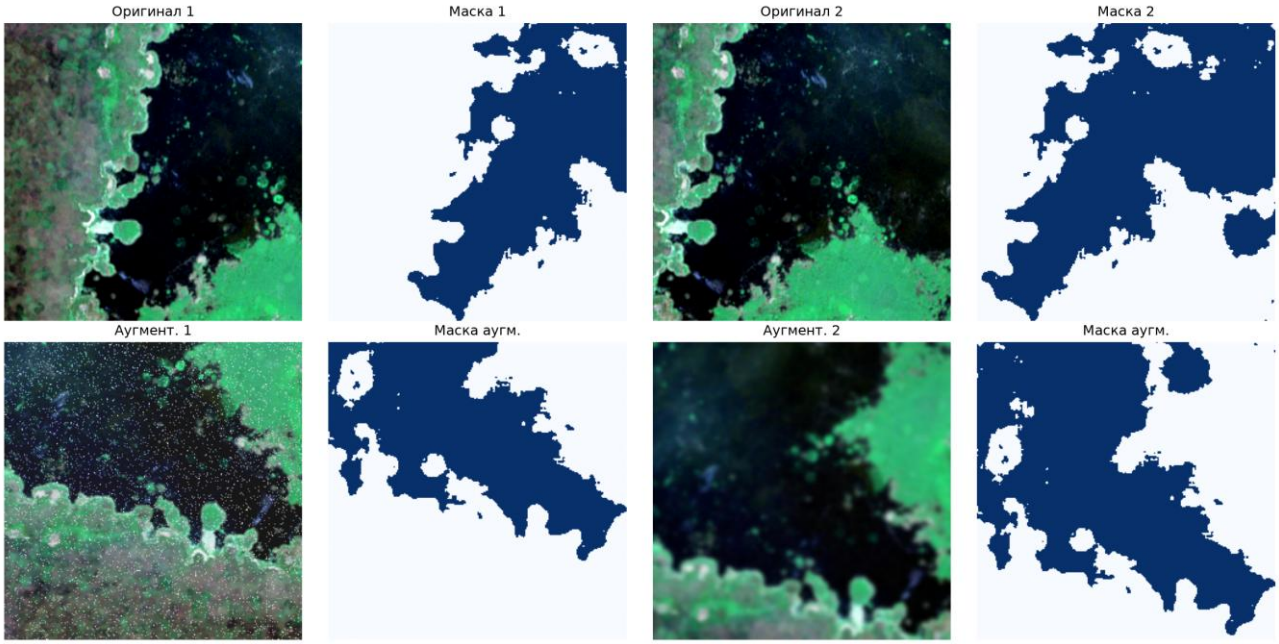


Рисунок 5 – Примеры аугментации 1

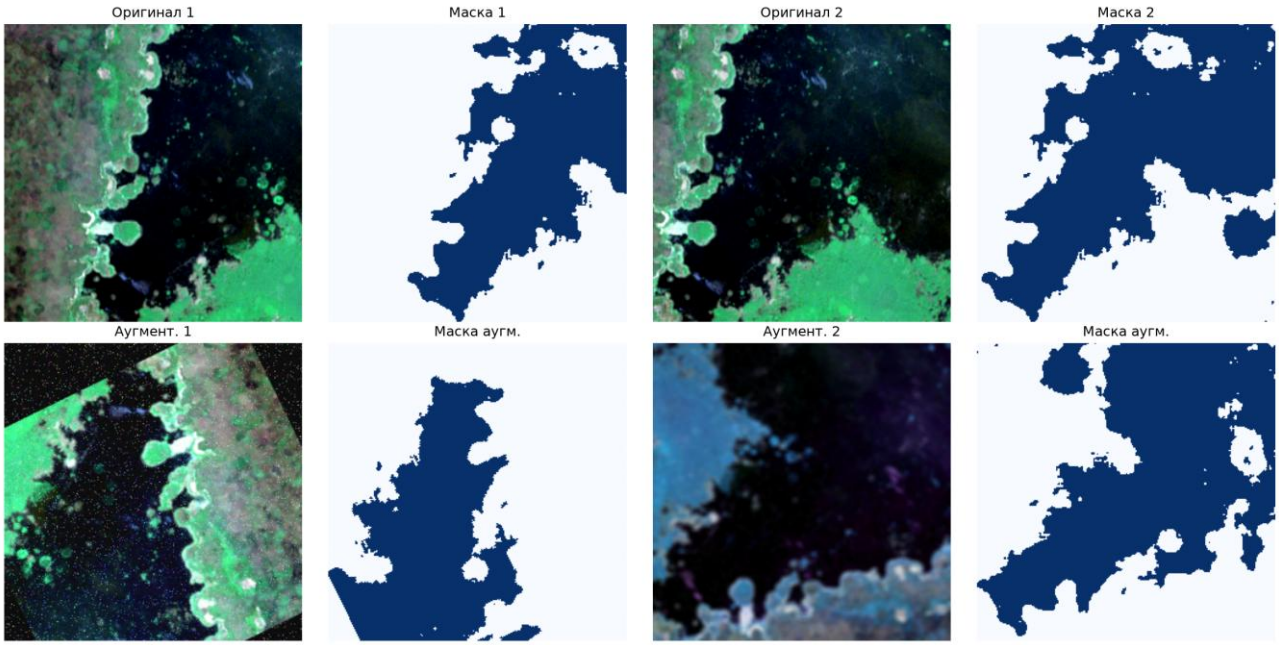


Рисунок 6 – Примеры аугментации 2

4. Реализация U-Net.

4.1-4.3 Архитектура U-Net реализована в классе UNet средствами Keras. Базовым элемент является сверточный блок conv_block из двух свёрток 3 на 3, каждая с *BatchNormalization* и активацией *ReLU*, плюс *Dropout*. Кодировщик состоит из четырёх таких блоков с *MaxPooling* 2 на 2. На каждом уровне размер карты признаков уменьшается вдвое, а число фильтров удваивается. В самой узкой части 512 фильтров. Декодировщик восстанавливает исходное разрешение транспонированными свёртками *Conv2DTranspose*. На каждом уровне к ним через присоединяются карты признаков кодировщика того же размера, что сохраняет мелкие детали. Выходной слой свёртка 1 на 1 с сигмоидой. Она даёт для каждого пикселя вероятность того, что это вода.

Реализованы три функции потерь: Dice Loss, Tversky Loss и комбинированная функция, которая складывает BCE и Tversky Loss.

Модель обучалась в Google Colab и Kaggle на GPU (Tesla T4). Использовались оптимизатор Adam и размер батча 8. К обучающей выборке применялась аугментация. Качество контролировалось по метрике IoU на валидационной выборке. Для управления обучением использовались три

колбэка: *ModelCheckpoint* сохранял модель с лучшим val IoU, *EarlyStopping* останавливал обучение, если val IoU не улучшался 30 эпох, а *ReduceLROnPlateau* уменьшал learning rate вдвое при застое val loss в течение 5 эпох.

Результаты обучения с разными функциями потерь при n_filters=16 и epochs=50 находятся в таблице 2.

Таблица 2 – Результаты обучения

Функция\Метрика	Train IoU	Val IoU
Dice Loss	0.8576	0.6252
Tversky Loss	0.6248	0.6250
Combined Loss	0.3898	0.4996

По данным таблицы 2 лучшие результаты на валидационной выборке показали Dice Loss и Tversky Loss, значения Val IoU у них практически одинаковые. Dice Loss имеет большой разрыв между обучающей и валидационной выборками и при этом значение Val IoU при обучении сильно колебалось, а лучший результат был получен только на последней, 50-й эпохе. Модель с Tversky Loss показала почти одинаковые значения IoU на обучающей и валидационной выборках, то есть она лучше обобщает данные и обучается стабильнее, а лучший результат был достигнут на 35-й эпохе. Combined Loss показала худший результат, лучшее значение было получено уже на 4-й эпохе, после чего качество на валидации резко снизилось, несмотря на рост Train IoU, и обучение было остановлено на 34-й эпохе механизмом ранней остановки.

5. Анализ ошибок и визуализация

5.1 U-Net выдаёт для каждого пикселя вероятность того, что это вода, и итоговая маска зависит от порога бинаризации. Для каждой обученной модели порог подбирался на валидационной выборке перебором значений от 0.3 до 0.8 с шагом 0,05. Критерием служил средний IoU после постобработки (см. рис. 7, 8 и 9).

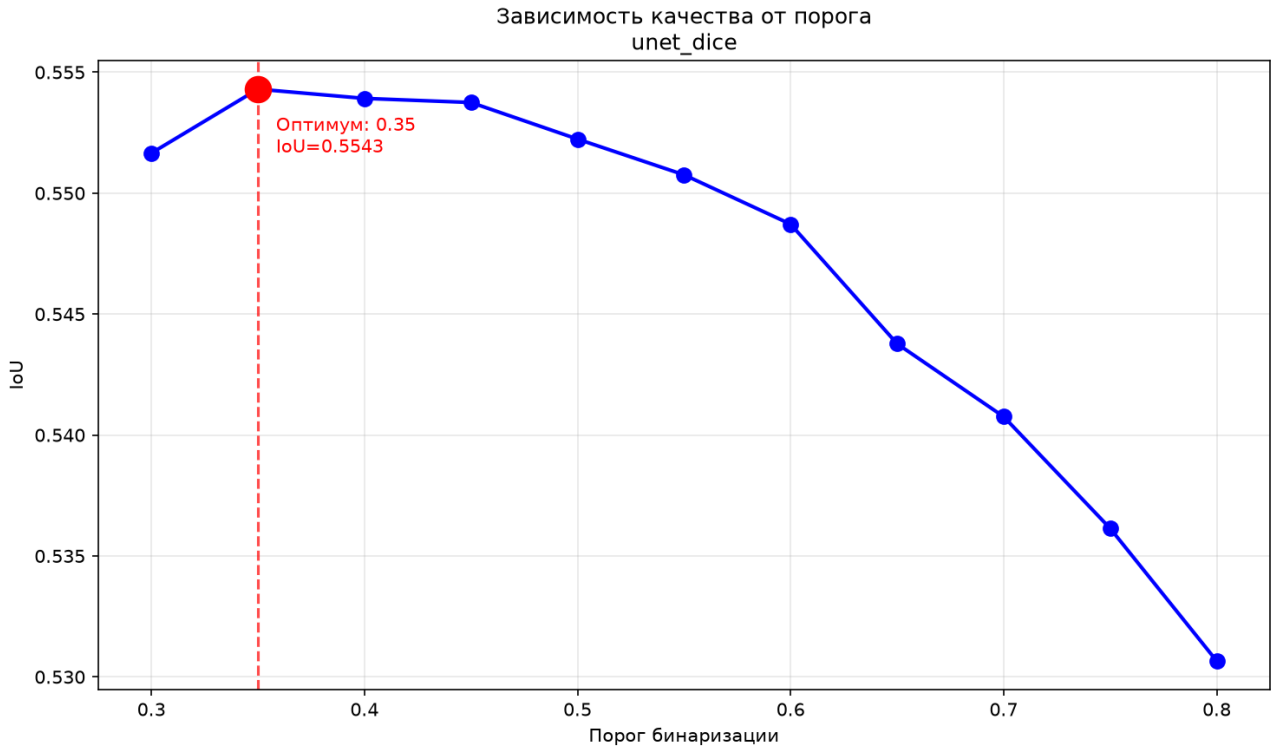


Рисунок 7 – Зависимость IoU от порога бинаризации для модели с Dice Loss

На рисунке 7 показана зависимость IoU от порога для модели с Dice Loss. Максимальное значение IoU 0.5543 достигается при пороге 0.35, при этом в диапазоне от 0.35 до 0.45 качество практически не меняется. При дальнейшем увеличении порога IoU плавно снижается. Общий разброс значений очень низкий, поэтому модель с Dice Loss слабо чувствительна к выбору порога.

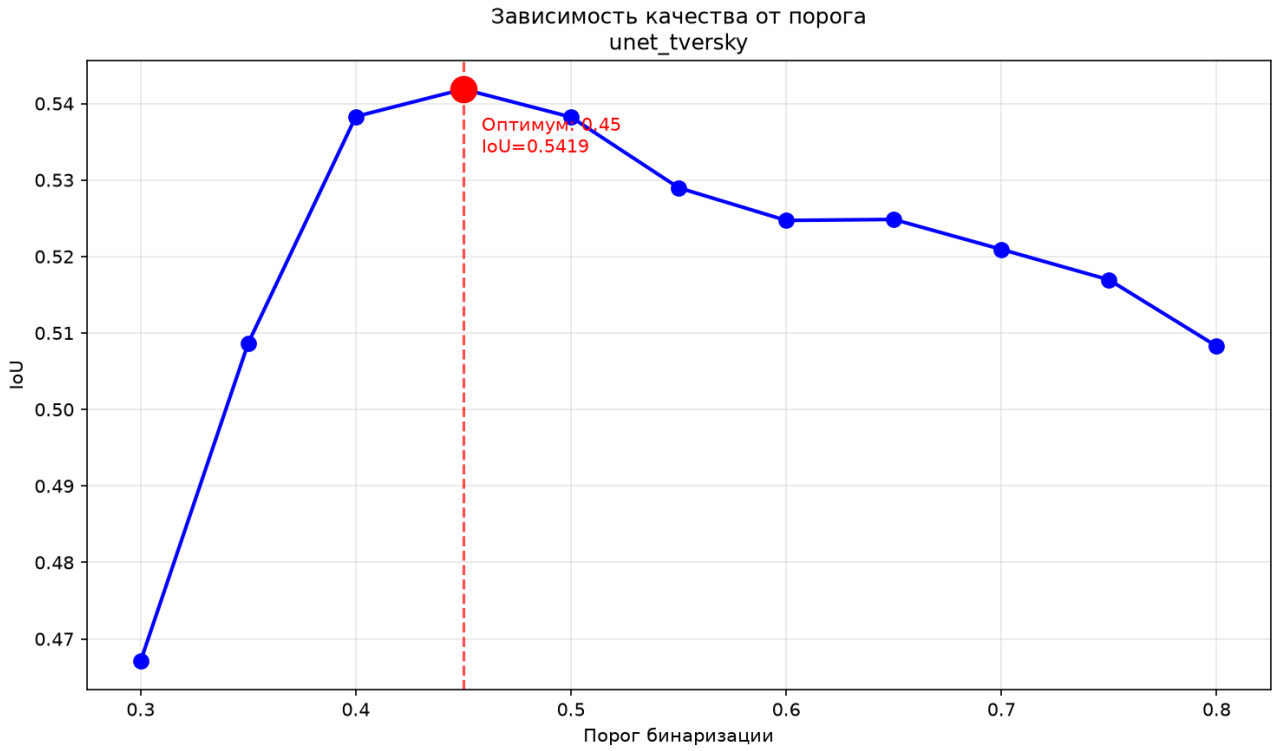


Рисунок 8 – Зависимость IoU от порога бинаризации для модели с Tversky Loss

На рисунке 8 показана зависимость IoU от порога для модели с Tversky Loss. Оптимальный порог равен 0.45 со значением IoU 0.5419. При низких порогах качество заметно хуже, так как модель относит к воде лишние пиксели. После оптимума IoU постепенно снижается. Чувствительность к порогу у этой модели выше, чем у модели с Dice Loss.

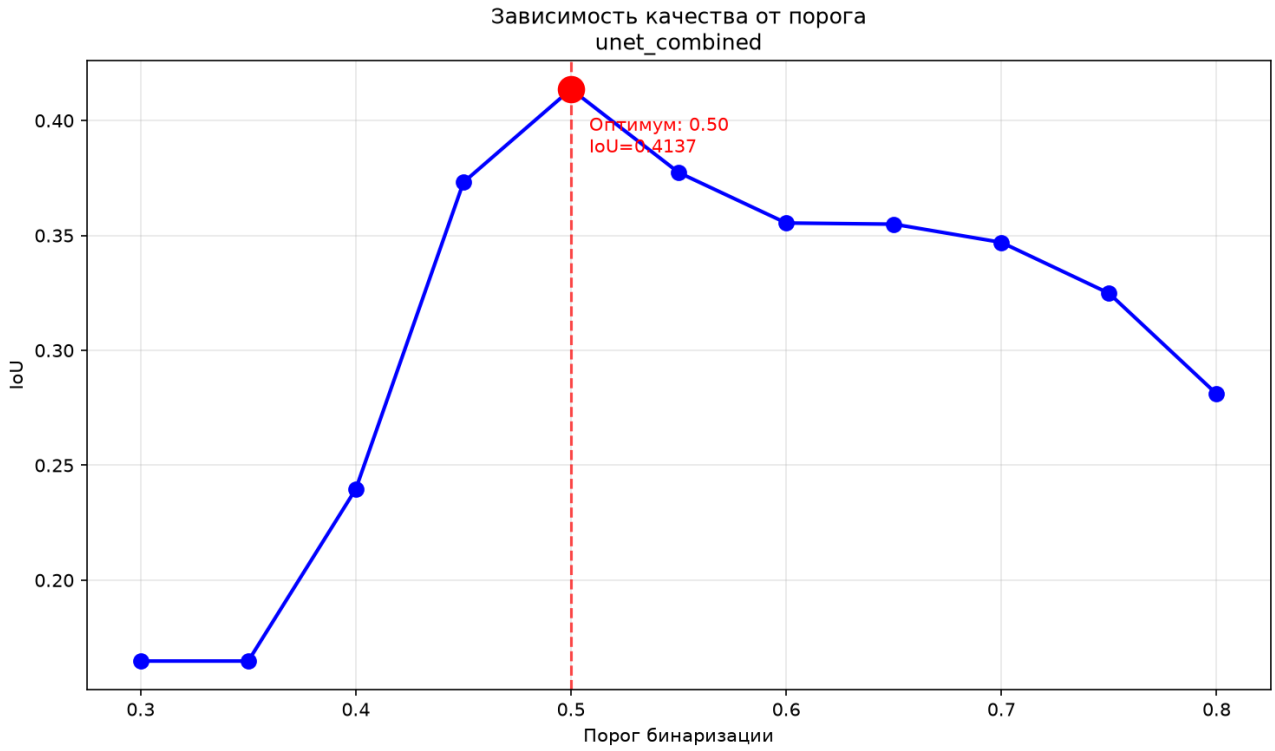


Рисунок 9 – Зависимость IoU от порога бинаризации для модели с Combined Loss

На рисунке 9 показана зависимость IoU от порога для модели с Combined Loss. Оптимальный порог равен 0.50 со значением IoU 0.4137. При низких порогах качество минимально, затем резко растёт до оптимума и снова снижается. Эта модель наиболее чувствительна к выбору порога, и даже при оптимальном пороге она значительно уступает двум другим моделям.

5.2 В Tversky Loss параметр α задаёт штраф за пропуск воды (FN), а β задает штраф за ложное срабатывание (FP). Чтобы оценить их влияние, обучено пять моделей U-Net с разными парами (α, β). Каждая модель обучалась 20 эпох на подвыборке из 100 обучающих патчей, а оценивалась на валидационной выборке по IoU, Recall, Precision и F1 (см. рис. 10 и таблицу 3).

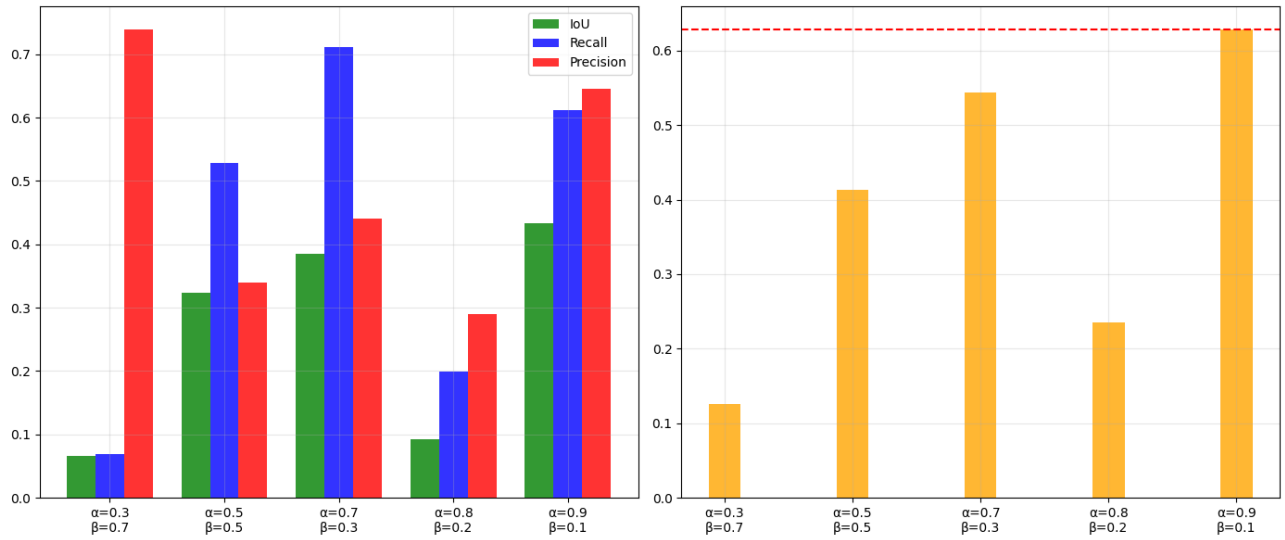


Рисунок 10 – Метрики качества при разных значениях параметров α и β

Таблица 3 – Метрики качества при разных значениях параметров α и β

α	β	IoU	Recall	Precision	F1
0.3	0.7	0.066099	0.068411	0.738645	0.125224
0.5	0.5	0.323815	0.528347	0.339077	0.413063
0.7	0.3	0.384515	0.710652	0.441079	0.544317
0.8	0.2	0.092091	0.198578	0.290424	0.235876
0.9	0.1	0.432851	0.612162	0.645071	0.628186

На рисунке 10 и в таблице 3 показано, как параметры α и β влияют на качество сегментации. При $\alpha = 0.3$ и $\beta = 0.7$ модель почти не выделяет воду: Precision высокий, но Recall очень низкий, поэтому IoU составляет всего 0.066. С увеличением α до 0.7 начинает расти Recall и IoU повышается до 0.385. Исключением стала пара $\alpha = 0.8$ и $\beta = 0.2$, при которой все метрики резко снизились. Наилучший результат получен при $\alpha = 0.9$ и $\beta = 0.1$, IoU имеет значение 0.433, F1 = 0.628, при этом Recall и Precision сбалансированы.

5.3 Для наглядного анализа ошибок предсказания моделей выведены на четырёх тестовых патчах. Для каждого патча показаны снимок, эталонная маска, маски трёх моделей с указанием порога и IoU, а также карта ошибок лучшей на

этом патче модели. На карте ошибок зелёным отмечены верно найденные пиксели воды (TP), красным отмечены ложные срабатывания (FP) и жёлтым отмечена пропущенная вода (FN). Чтобы показать эффект подбора порогов, построены две визуализации на одних и тех же патчах: с эмпирическими порогами (см. рис. 11) и с порогами, найденными на валидационной выборке (пункт 5.1) (см. рис. 12).



Рисунок 11 – Результаты сегментации на тестовых патчах при эмпирических порогах



Рисунок 12 – Результаты сегментации на тестовых патчах при порогах на валидационной выборке

На рисунках 11 и 12 видно, что модели с Dice Loss и Tversky Loss хорошо выделяют крупные водные объекты. Основные ошибки возникают на сложных участках. На первом и третьем патчах модели пропускают мелководные и заросшие растительностью участки воды и ошибочно относят к воде прибрежные участки суши. С мелкими водоёмами качество заметно ниже у всех моделей, так как небольшие объекты либо пропускаются, либо выделяются неточно. Модель с Combined Loss практически не выделяет воду, на всех патчи её маска почти пустая, а IoU близок к нулю.

6. Интеграция в пайплайн

6.1 Для применения обученной модели к снимкам создан класс SegmentationPipeline. Он наследует пайплайн предыдущих работ и добавляет методы сегментации:

- *load_segmentation_model* загружает обученную модель U-Net без функций потерь, так как они нужны только при обучении.
- *load_best_model* читает из файла имя лучшей модели и её оптимальный порог.
- *segment_image* сегментирует снимок произвольного размера.
- *mask_to_geojson* переводит растровую маску в векторный слой

Работа класса проверена на снимке Sentinel-2 дельты Волги из ПР№ 2 (см. рис. 13). Снимок был разрезан на патчи и в результате получены растровая маска воды full_mask.tif и векторный слой water_bodies.geojson.



Рисунок 13 – Сегментированная маска снимка

На рисунке 13 модель выделила крупный водоём в северной части снимка, и его контур хорошо совпадает с эталонной маской. Также выделен узкий линейный канал, пересекающий снимок с севера на юг. При этом модель отнесла к воде несколько крупных участков, которые на эталонной маске относятся к суше, заболоченной или с густой тёмной растительностью. Это ложные срабатывания, которые завышают площадь воды. Кроме того, у некоторых объектов видны прямые горизонтальные и вертикальные границы, что связано со склейкой снимка из отдельных патчей. В целом модель хорошо находит крупные водоёмы, но ошибается на заболоченных участках и плохо выделяет узкие водные объекты.

6.2 Для сохранения результатов работы в класс *SegmentationPipeline* добавлен метод *save_pract34_results*. Сначала для каждой модели на тестовой выборке рассчитан средний IoU с её оптимальным порогом, подобранным на валидационной выборке.

Выполнение самостоятельных заданий.

* 2.2 Для анализа ошибок пороговых методов выбран патч с долей воды 20,4% (см. рис. 14). Для него построены карты и матрицы ошибок (см. рис. 15 и таблица 4).

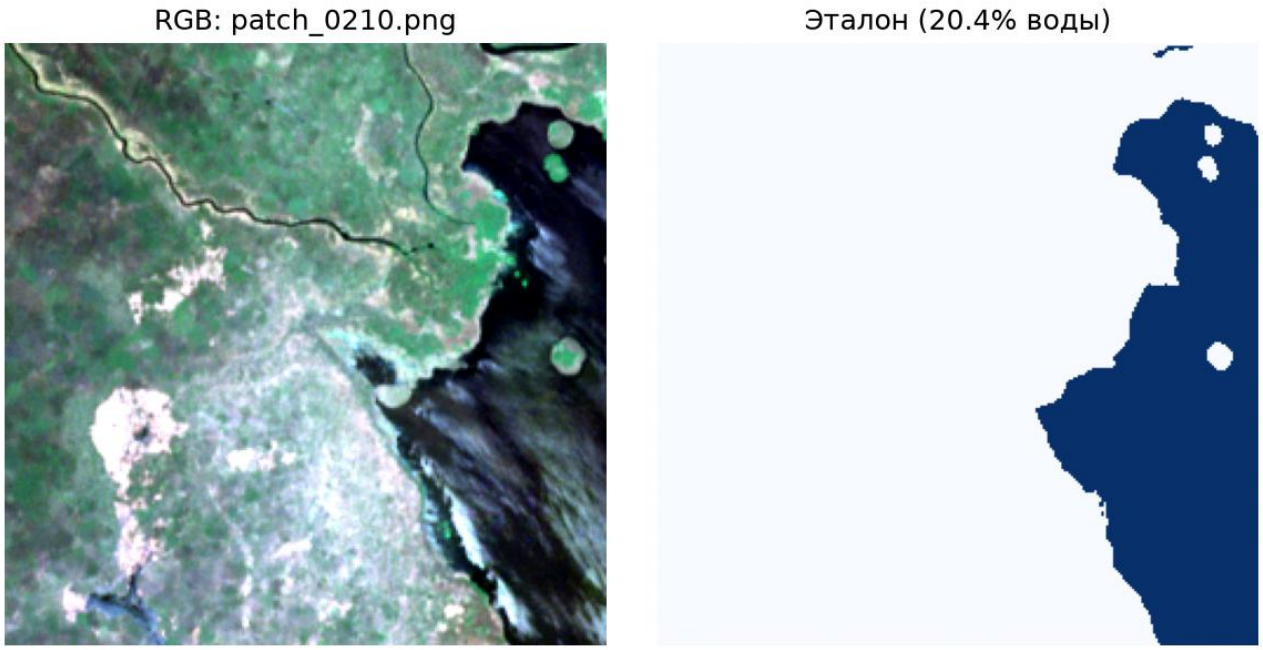


Рисунок 14 – Патч и эталонная маска

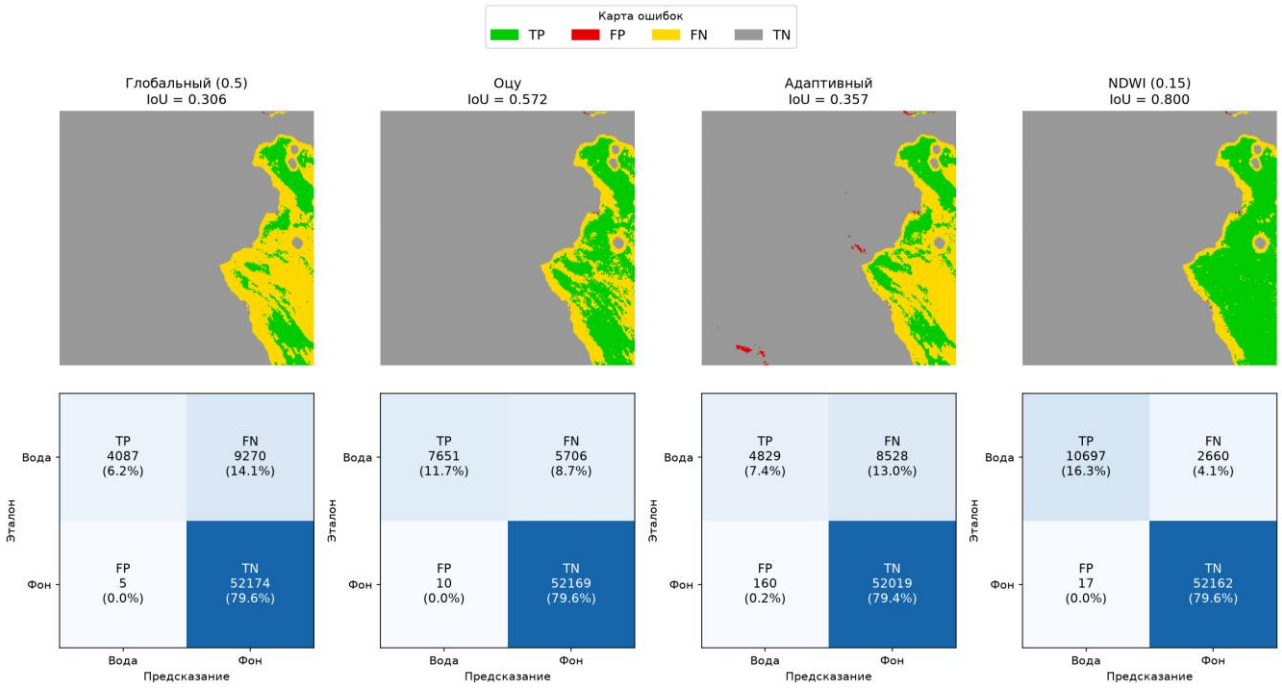


Рисунок 15 – Матрицы ошибок

Таблица 4 – Полученные метрики

Метод\Метрика	IoU	Precision	Recall
Глобальный (0.5)	0.306	0.999	0.306
Оцу	0.572	0.999	0.573

Адаптивный	0.357	0.968	0.362
NDWI (0.15)	0.800	0.998	0.801

Каждый пиксель отнесён к одной из групп:

- TP – верно найденная вода (зелёный);
- FP – ложная вода (красный);
- FN – пропущенная вода (жёлтый);
- TN – верно определённый фон (серый);

Около 80% пикселей патча относятся к TN, поэтому основной метрикой выбран IoU. Лучший результат у порога NDWI, затем идет метод Оцу, адаптивный порог и глобальный порог. Во всех методах преобладают ошибки FN (пропуски воды). Ложных срабатываний почти нет. Поэтому Precision имеет значение около 0.9, а Recall варьируется от 0.3 до 0.8, из-за пропущенных пикселей воды. У всех методов пропуски идут полосой вдоль береговой линии, где отражение смешанное. У глобального, адаптивного порогов и метода Оцу пропуски есть и внутри водоёма, где значение NDWI понижено.

Общее преобладание FN показывает, что пороговые методы работают консервативно. Пороги слишком строги для мутной воды и мелководных объектов. Таким образом выбраны значения переменных, что $\alpha > \beta$ в Tversky Loss при обучении U-Net, так как пропуски воды должны штрафоваться сильнее ложных срабатываний.

* 6.3 В метод *segment_and_export* добавлен автоматический расчёт статистики водных объектов. Площади полигонов считаются в UTM, результат сохраняется в water_bodies_stats.json. На снимке дельты Волги модель U-Net выделила 177 водоёмов общей площадью 42 512 500 м^2 (3 374.7 га), средний размер объекта 363 355 м^2 (36.34 га).

Для эколога общая площадь воды на снимках разных дат показывает, как меняется водная поверхность территории, а именно разливы, обмеление и зарастание водоёмов. Если число водоёмов растёт, а их средний размер

уменьшается, водная сеть распадается на мелкие участки, и такие участки могут скорее всего пересохнуть. Площади в гектарах можно напрямую сравнивать с данными водного реестра и выявлять водоёмы, у которых площадь сократилась.

*** 6.3 Для анализа динамики водной поверхности в класс *SegmentationPipeline* добавлен метод *segment_time_series*. На вход он принимает список снимков одной территории и общую область интереса. Каждый снимок сегментируется лучшей моделью U-Net. В результате получаем GeoDataFrame с датой, площадью воды (в км^2), числом водных объектов и их геометрией. Метод применён к пяти малооблачным снимкам Sentinel-2 с января по сентябрь 2025 года. Область интереса представляет собой прямоугольник площадью, охватывающий мелководное озеро в северной части снимка (см. рис. 14).



Рисунок 14 – Область интереса на разных снимках

Для проверки на график динамики нанесены площади воды по маскам Dynamic World (см. рис. 15).

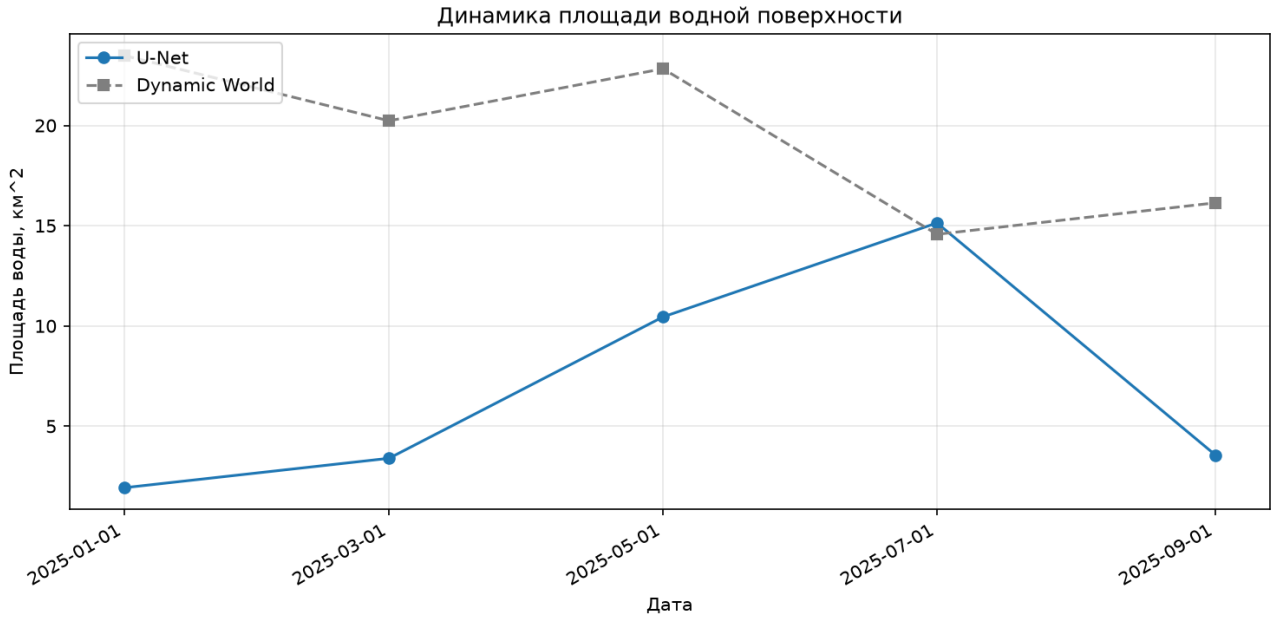


Рисунок 15 – Динамика изменения водной поверхности

По истинным маскам из Dynamic World площадь воды составляет 20-23 км^2 с мая по июль сокращается до около 15 км^2 и 16 км^2 в сентябре. Это соответствует летнему обмелению и зарастанию озера, которое заметно и на июльском снимке. Модель U-Net близка к эталону только в июле. На остальных снимках она сильно занижает площадь. Модель распознаёт только тёмную воду, а светлую мелкую и мутную воду, а также лёд относит к суше. Причина в том, что модель обучена на одном снимке из ПР№2, а также в том, что цвет водной поверхности сильно меняется в зависимости от сезона, мутности и ледового покрова. Чтобы модель годилась для временного мониторинга, её нужно обучать на снимках разных сезонов, а также добавить ближний и коротковолновый инфракрасные каналы, в которых вода отличается от суши независимо от цвета.

Выводы.

Освоены практические навыки сегментации водных объектов на спутниковых снимках, от классических пороговых методов до нейросетевой архитектуры U-Net. Среди пороговых методов лучший результат показал порог по индексу NDWI, а анализ ошибок показал, что пороговые методы в основном пропускают воду вдоль береговой линии и на мутных мелководьях. Для

обучения U-Net подготовлен датасет патчей с эталонными масками Dynamic World, разделённый на выборки для исключения утечки данных, и реализована аугментация. Модели с Dice Loss и Tversky Loss показали одинаковое качество на валидации, а Combined Loss значительно им уступил. Подбор порога бинаризации на валидационной выборке повысил стабильность результатов, а исследование параметров Tversky Loss показало, что лучший результат достигается при $\alpha = 0.9$ и $\beta = 0.1$. Модели хорошо выделяют крупные водоёмы, но ошибаются на заболоченных участках и плохо выделяют узкие и мелкие объекты. Создан класс *SegmentationPipeline*, который сегментирует снимок произвольного размера и экспортирует результат в GeoJSON со статистикой водных объектов. Анализ снимков разных месяцев года показал, что модель обученная на одном летнем снимке, сильно занижает площадь воды в другие сезоны, поэтому для мониторинга её необходимо обучать на снимках разных сезонов.

ПРИЛОЖЕНИЕ А

Исходный код программы

Файл main.py:

```
import glob
import json
import os
import matplotlib.pyplot as plt
import numpy as np
import rasterio
import tensorflow as tf
from shapely.geometry import box
from segmentation_pipeline import SegmentationPipeline
from water_quality_pipeline_extended import WaterQualityPipelineExtended


MODELS = {'Dice Loss': 'unet_dice', 'Tversky Loss': 'unet_tversky', 'Combined Loss': 'unet_combined'}

def task_2_1(pipeline: WaterQualityPipelineExtended):
    pipeline.test_methods_on_patches(n_samples=3)


def task_2_2(pipeline: WaterQualityPipelineExtended):
    pipeline.plot_confusion_matrix()


def task_3_1(pipeline: WaterQualityPipelineExtended):
    pipeline.create_dataset_from_pract2(stride=64)
    pipeline.visualize_dataset(n_samples=3)


def task_3_3(pipeline: WaterQualityPipelineExtended):
    (X_train, y_train), (X_val, y_val), (X_test, y_test) = pipeline.load_dataset()
```

```
        # pipeline.plot_dataset_split()
        # pipeline.visualize_augmentation(n_samples=2)


    def task_5_1(pipeline: WaterQualityPipelineExtended):
        pipeline.load_dataset()
        X_val, y_val = pipeline.dataset['X_val'],
pipeline.dataset['y_val']
        models, thresholds = {}, {}
        for name, file in MODELS.items():
            models[name] =
tf.keras.models.load_model(f'{pipeline.train_dir}/{file}.keras',
compile=False)
            thresholds[name], _ =
pipeline.find_optimal_threshold(models[name], X_val, y_val,
model_name=file, thresholds=np.arange(0.3, 0.85, 0.05))
        return models, thresholds


    def task_5_3(pipeline: WaterQualityPipelineExtended):
        models, thresholds = task_5_1(pipeline)
        X_test, y_test = pipeline.dataset['X_test'],
pipeline.dataset['y_test']
        np.random.seed(42)
        pipeline.visualize_predictions(models, X_test, y_test,
n_samples=4)
        np.random.seed(42)
        pipeline.visualize_predictions_with_thresholds(models,
X_test, y_test, thresholds, n_samples=4)


    def task_6_1(test_image='../../lb2/src/data/delta_volga.tif',
                 output_mask='data/full_mask.tif',
                 output_geojson='data/water_bodies.geojson'):
        seg_pipeline = SegmentationPipeline()
```

```
        info_path =
f'{seg_pipeline.train_dir}/best_model_info.json'

        if not os.path.exists(info_path):
            seg_pipeline.load_dataset()
            X_val, y_val = seg_pipeline.dataset['X_val'],
seg_pipeline.dataset['y_val']
            best = None
            for file in MODELS.values():
                model =
tf.keras.models.load_model(f'{seg_pipeline.train_dir}/{file}.keras
', compile=False)
                threshold, iou =
seg_pipeline.find_optimal_threshold(model, X_val, y_val,
model_name=file, thresholds=np.arange(0.3, 0.85, 0.05))
                if best is None or iou > best[2]:
                    best = (file, threshold, iou)
            with open(info_path, 'w', encoding='utf-8') as f:
                json.dump({'best_model_file': f'{best[0]}.keras',
                           'best_threshold': round(float(best[1]),
2),
                           'val_iou': round(float(best[2]), 4)},
f, ensure_ascii=False, indent=2)

        seg_pipeline.load_best_model(info_path)

        seg_pipeline.segment_image(
            image_path=test_image,
            output_path=output_mask,
            threshold=seg_pipeline.best_threshold
        )

        seg_pipeline.mask_to_geojson(
            mask_path=output_mask,
            output_path=output_geojson,
```

```
            min_area=1000
        )

        with rasterio.open(output_mask) as src:
            mask = src.read(1)

        plt.figure(figsize=(10, 10))
        plt.imshow(mask, cmap='Blues')
        plt.title('Сегментированная маска всего снимка')
        plt.axis('off')
        plt.savefig(seg_pipeline.output_dir + 'full_mask.png',
dpi=150, bbox_inches='tight')
        plt.show()


    def task_6_2():
        seg_pipeline = SegmentationPipeline()
        models, thresholds = task_5_1(seg_pipeline)
        X_test, y_test = seg_pipeline.dataset['X_test'],
seg_pipeline.dataset['y_test']

        results = {}
        for name, model in models.items():
            probs = model.predict(X_test, batch_size=4,
verbose=0)[..., 0]
            metrics = [seg_pipeline.segmenter.calculate_metrics(
                            y_test[i],
seg_pipeline.segmenter.postprocess_mask((probs[i] >
thresholds[name]).astype(np.float32)))
                       for i in range(len(y_test))]
            results[name] = {'IoU': float(np.mean([m['IoU'] for m
in metrics])),
                             'threshold':
round(float(thresholds[name]), 2)}
```

```
            print(f"{name:15}:      IoU      на      test      =
{results[name]['IoU']:.4f}                                    (порог
{results[name]['threshold']:.2f})")

        seg_pipeline.save_pract34_results(models, results,

seg_pipeline.dataset['X_train'],     seg_pipeline.dataset['X_val'],
X_test)


    def task_6_3_1(test_image='data/delta_volga.tif',
                   output_mask='data/full_mask.tif',
                   output_geojson='data/water_bodies.geojson'):
        seg_pipeline = SegmentationPipeline()

seg_pipeline.load_best_model(f'{seg_pipeline.train_dir}/best_model
_info.json')
        return         seg_pipeline.segment_and_export(test_image,
output_mask=output_mask,

output_geojson=output_geojson, min_area=1000)


    def task_6_3_2(series_dir='data/date_series'):
        seg_pipeline = SegmentationPipeline()

seg_pipeline.load_best_model(f'{seg_pipeline.train_dir}/best_model
_info.json')

        image_paths                                              =
sorted(glob.glob(f'{series_dir}/delta_volga(*).tif'))
        reference_paths                                          =
sorted(glob.glob(f'{series_dir}/dynamic_world_label(*).tif'))

        top_left = (45.801636, 48.603861)
```

```
        bottom_right = (45.749001, 48.694187)

        roi = box(top_left[1], bottom_right[0], bottom_right[1],
top_left[0])
        seg_pipeline.plot_roi(image_paths, roi)

        gdf = seg_pipeline.segment_time_series(image_paths, roi,
reference_paths=reference_paths)
        gdf.to_file('data/water_time_series.geojson',
driver='GeoJSON')
        return gdf


    def main():
        pipeline = WaterQualityPipelineExtended()
        # task_3_1(pipeline)
        # task_2_1(pipeline)
        # task_2_2(pipeline)
        # task_3_3(pipeline)
        # task_5_1(pipeline)
        # task_5_3(pipeline)
        # task_6_1()
        # task_6_2()
        # task_6_3_1()
        # task_6_3_2()


    if __name__ == '__main__':
        main()
```

Файл segmentation_pipeline.py:

```
import json
import os
import pickle
import re
```

```
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import rasterio
import tensorflow as tf
from rasterio import features
from rasterio.enums import Resampling
from rasterio.features import geometry_mask
from shapely.geometry import MultiPolygon, shape

from water_quality_pipeline_extended import WaterQualityPipelineExtended


class SegmentationPipeline(WaterQualityPipelineExtended):
    """
    Расширение пайплайна методами сегментации для работы с новыми снимками.
    Позволяет загрузить обученную модель, применить ее к любому спутниковому снимку
    и сохранить результат в виде GeoTIFF и GeoJSON.
    """

    def __init__(self, name="Интеграция сегментации"):
        super().__init__(name)
        self.segmentation_model = None
        self.segmentation_masks = None
        self.best_threshold = None

    def load_segmentation_model(self, model_path='model_train/unet_dice.keras'):
        """
        Загрузка обученной модели сегментации.
```

```
            compile=False: функции потерь нужны только для
обучения,
            для предсказания модель загружается без них
            """
            self.segmentation_model                                =
tf.keras.models.load_model(model_path, compile=False)
            print(f"Модель загружена: {model_path}")
            return self.segmentation_model

        def                                  load_best_model(self,
info_path='model_train/best_model_info.json'):
            """
            Загружает лучшую модель и ее оптимальный порог из JSON-
файла
            вида     {"best_model_file":     "unet_tversky.keras",
"best_threshold": 0.45}
            """
            with open(info_path, 'r', encoding='utf-8') as f:
                best_info = json.load(f)


self.load_segmentation_model(f'{self.train_dir}/{best_info["best_m
odel_file"]}')
            self.best_threshold = best_info['best_threshold']
            print(f"Оптимальный    порог    для    этой    модели:
{self.best_threshold:.2f}")
            return self.best_threshold

        def   segment_image(self,   image_path,   output_path=None,
patch_size=256, overlap=0.5, threshold=None):
            """
            Сегментация спутникового снимка скользящим окном.
            Снимок режется на патчи с перекрытием overlap (доля от
размера патча),
```

```
            предсказания всех патчей считаются батчами и
усредняются в местах перекрытия.
            threshold=None - используется best_threshold (если
загружен) или 0.5
            """
            if self.segmentation_model is None:
                raise RuntimeError("Сначала загрузите модель:
load_segmentation_model()")
            if threshold is None:
                threshold = self.best_threshold if
self.best_threshold is not None else 0.5

            with rasterio.open(image_path) as src:
                img =
np.stack([self.norm_band(src.read(i).astype(np.float32)) for i in
(1, 2, 3)],
                                 axis=-
1).astype(np.uint8).astype(np.float32) / 255.0
                crs, transform = src.crs, src.transform

            h, w = img.shape[:2]
            step = max(1, int(patch_size * (1 - overlap)))

            y_coords = list(range(0, h - patch_size + 1, step))
            x_coords = list(range(0, w - patch_size + 1, step))
            if y_coords[-1] != h - patch_size:
                y_coords.append(h - patch_size)
            if x_coords[-1] != w - patch_size:
                x_coords.append(w - patch_size)

            patches = np.array([img[y:y + patch_size, x:x +
patch_size] for y in y_coords for x in x_coords])

            print(f"Обработка {len(patches)} патчей...")
```

```
            predictions = self.segmentation_model.predict(patches,
batch_size=16, verbose=1)[..., 0]

            mask_accum = np.zeros((h, w), dtype=np.float32)
            weight_accum = np.zeros((h, w), dtype=np.float32)
            coords = [(y, x) for y in y_coords for x in x_coords]
            for (y, x), pred in zip(coords, predictions):
                mask_accum[y:y + patch_size, x:x + patch_size] +=
pred
                weight_accum[y:y + patch_size, x:x + patch_size] +=
1.0

            self.segmentation_masks      =      mask_accum      /
np.maximum(weight_accum, 1e-8)

            if output_path:
                binary = self.segmenter.postprocess_mask(
                    (self.segmentation_masks                      >
threshold).astype(np.float32)).astype(np.uint8)
                with rasterio.open(
                    output_path, 'w',
                    driver='GTiff',
                    height=h,
                    width=w,
                    count=1,
                    dtype=np.uint8,
                    crs=crs,
                    transform=transform,
                    tiled=True, blockxsize=256, blockysize=256,
                    compress='lzw', photometric='palette'
                ) as dst:
                    dst.write(binary, 1)
                    dst.write_colormap(1, {0: (0, 0, 0, 255), 1:
(255, 255, 255, 255)})
```

```
                    dst.build_overviews([2,         4,         8],
Resampling.nearest)
                print(f"Маска   сохранена:   {output_path}   (порог
{threshold:.2f})")

            return self.segmentation_masks

        def           mask_to_geojson(self,           mask_path,
output_path='water_bodies.geojson', min_area=1000):
            """
            Конвертация бинарной маски (GeoTIFF) в векторный слой
GeoJSON.
            Фильтрует полигоны по минимальной площади (в квадратных
метрах).

            Параметры:
            - mask_path: путь к GeoTIFF с бинарной маской (0 или
1).
            - output_path: путь для сохранения GeoJSON.
            - min_area: минимальная площадь полигона в кв. метрах
              (площадь считается в системе координат снимка, она
должна быть метровой, например UTM).

            Возвращает:
            - gdf: GeoDataFrame с полигонами водных объектов.
            """
            with rasterio.open(mask_path) as src:
                mask = src.read(1)
                transform = src.transform
                crs = src.crs

            results = []
            for geom, _ in features.shapes(mask.astype(np.int16),
mask=mask == 1, transform=transform):
                geom_shapely = shape(geom)
```

```
                if geom_shapely.area >= min_area:
                    results.append(geom_shapely)

            gdf = gpd.GeoDataFrame(geometry=results, crs=crs)
            gdf['area_m2'] = gdf.geometry.area
            gdf = gdf.to_crs('EPSG:4326')
            gdf.to_file(output_path, driver='GeoJSON')
            print(f"GeoJSON сохранен: {output_path} (объектов:
{len(gdf)})")
            return gdf

        def segment_and_export(self, image_path,
output_mask='segmentation_mask.tif',

output_geojson='water_bodies.geojson',
                               threshold=None, min_area=1000):
            """
            Полный цикл: сегментация снимка, экспорт в GeoTIFF и
GeoJSON и расчет статистики
            водных объектов. Статистика сохраняется рядом с GeoJSON
(*_stats.json)
            """
            self.segment_image(image_path,
output_path=output_mask, threshold=threshold)
            gdf = self.mask_to_geojson(output_mask,
output_geojson, min_area=min_area)
            print("Интеграция завершена: получены растровая и
векторная маски.")

            stats = self.calculate_water_statistics(gdf)
            stats_path = os.path.splitext(output_geojson)[0] +
'_stats.json'
            with open(stats_path, 'w', encoding='utf-8') as f:
                json.dump(stats, f, ensure_ascii=False, indent=2)
            print(f"Статистика сохранена: {stats_path}")
```

```
            return stats

        def calculate_water_statistics(self, gdf):
            """
            Статистика водных объектов по GeoDataFrame из
mask_to_geojson
            (площадь берется из колонки area_m2, посчитанной в
метровой системе координат снимка)
            1 га = 10 000 м²
            """
            total_m2 = float(gdf['area_m2'].sum())
            n_objects = len(gdf)
            mean_m2 = total_m2 / n_objects if n_objects else 0.0

            stats = {
                'total_area_m2': round(total_m2, 1),
                'total_area_ha': round(total_m2 / 1e4, 2),
                'n_objects': n_objects,
                'mean_area_m2': round(mean_m2, 1),
                'mean_area_ha': round(mean_m2 / 1e4, 2),
            }
            print("\nСтатистика водных объектов:")
            print(f"Общая площадь: {stats['total_area_m2']:,.0f}
м2 ({stats['total_area_ha']:,.2f} га)".replace(',', ' '))
            print(f"Количество водоемов: {n_objects}")
            print(f"Средний размер объекта:
{stats['mean_area_m2']:,.0f} м2 ({stats['mean_area_ha']:,.2f}
га)".replace(',', ' '))
            return stats

        def plot_roi(self, image_paths, roi, dates=None,
filename='roi.png'):
            """
            Область интереса (красный контур) поверх RGB-снимков
серии.
```

```
            roi - shapely-геометрия в EPSG:4326 или
GeoDataFrame/GeoSeries
            """

            if dates is None:
                dates = [re.search(r'\d{4}-\d{2}-\d{2}',
os.path.basename(p)).group() for p in image_paths]
            roi = gpd.GeoSeries([roi], crs='EPSG:4326') if not
isinstance(roi, (gpd.GeoSeries, gpd.GeoDataFrame)) else roi

            fig, axes = plt.subplots(1, len(image_paths),
figsize=(5 * len(image_paths), 5.5), squeeze=False)
            for ax, path, date in zip(axes[0], image_paths, dates):
                with rasterio.open(path) as src:
                    rgb =
np.stack([self.norm_band(src.read(i).astype(np.float32)) for i in
(1, 2, 3)],
                                     axis=-1).astype(np.uint8)
                    b, crs = src.bounds, src.crs
                ax.imshow(rgb, extent=(b.left, b.right, b.bottom,
b.top))
                roi.to_crs(crs).boundary.plot(ax=ax, color='red',
linewidth=2)
                ax.set_xlim(b.left, b.right)
                ax.set_ylim(b.bottom, b.top)
                ax.set_title(date)
                ax.axis('off')

            roi_area = roi.to_crs(crs).area.sum() / 1e6
            fig.suptitle(f'Область интереса, площадь
{roi_area:.1f} км^2', fontsize=14, y=1.03)
            plt.tight_layout()
            plt.savefig(self.output_dir + filename, dpi=150,
bbox_inches='tight')
            plt.show()
```

```
        def segment_time_series(self, image_paths, roi,
dates=None, reference_paths=None,
                                threshold=None,
filename='water_dynamics.png'):
            """
            Сегментация серии снимков одной территории за разные
даты.

            Параметры:
            - image_paths: список путей к снимкам (GeoTIFF, каналы
B4, B3, B2, ...)
            - roi: общая область интереса (shapely-геометрия в
EPSG:4326 или GeoDataFrame/GeoSeries)
            - dates: список дат; если None - дата берется из имени
файла (первая дата вида YYYY-MM-DD)
            - reference_paths: необязательные маски Dynamic World
для сравнения (класс 0 - вода)
            - threshold: порог бинаризации (None - best_threshold
модели)

            Возвращает:
            - gdf: GeoDataFrame с колонками date, water_area (км²),
n_objects
              и геометрией водных объектов внутри ROI на каждую
дату
            """

            if threshold is None:
                threshold = self.best_threshold if
self.best_threshold is not None else 0.5
            if dates is None:
                dates = [re.search(r'\d{4}-\d{2}-\d{2}',
os.path.basename(p)).group() for p in image_paths]
```

```
            roi = gpd.GeoSeries([roi], crs='EPSG:4326') if not
isinstance(roi, (gpd.GeoSeries, gpd.GeoDataFrame)) else roi

            rows = []
            for i, (path, date) in enumerate(zip(image_paths,
dates)):
                print(f"\n[{i + 1}/{len(image_paths)}] {date}:
{os.path.basename(path)}")
                with rasterio.open(path) as src:
                    crs, transform, shape_hw = src.crs,
src.transform, (src.height, src.width)
                pixel_area = abs(transform.a * transform.e)

                roi_mask =
~geometry_mask(roi.to_crs(crs).geometry, out_shape=shape_hw,
transform=transform)

                probs = self.segment_image(path,
threshold=threshold)
                water = self.segmenter.postprocess_mask((probs >
threshold).astype(np.float32)).astype(np.uint8)
                water[~roi_mask] = 0

                polygons = [shape(g) for g, _ in
features.shapes(water.astype(np.int16), mask=water == 1,
transform=transform)]

                row = {'date': pd.to_datetime(date),
                       'water_area': sum(p.area for p in polygons)
/ 1e6,
                       'n_objects': len(polygons),
                       'geometry': MultiPolygon(polygons) if
polygons else None}

                if reference_paths is not None:
```

```
                    with rasterio.open(reference_paths[i]) as ref:
                        ref_water = (ref.read(1) == 0) & roi_mask
                    row['water_area_dw']   =   ref_water.sum()   *
pixel_area / 1e6

                rows.append(row)
                print(f"Площадь воды: {row['water_area']:.2f} км2,
объектов: {row['n_objects']}")

            gdf   =   gpd.GeoDataFrame(rows,   geometry='geometry',
crs=crs).sort_values('date').reset_index(drop=True)

            fig, ax1 = plt.subplots(figsize=(10, 5))
            ax1.plot(gdf['date'],     gdf['water_area'],     'o-',
color='tab:blue', label='U-Net')
            if 'water_area_dw' in gdf:
                ax1.plot(gdf['date'], gdf['water_area_dw'], 's--',
color='tab:gray', label='Dynamic World')
            ax1.set_ylabel('Площадь воды, км^2')
            ax1.set_xlabel('Дата')
            ax1.set_xticks(gdf['date'])
            ax1.set_xticklabels(gdf['date'].dt.strftime('%Y-%m-
%d'))
            ax1.grid(alpha=0.3)
            ax1.legend(loc='upper left')
            ax1.set_title('Динамика площади водной поверхности')
            fig.autofmt_xdate()
            plt.tight_layout()
            plt.savefig(self.output_dir   +   filename,   dpi=150,
bbox_inches='tight')
            plt.show()


print(gdf.drop(columns='geometry').to_string(index=False))
            return gdf
```

```
        def save_pract34_results(self, models, results, X_train,
X_val, X_test):
            """
            Сохранение результатов практической работы №3-4.

            Параметры:
            - models: словарь обученных моделей {имя: модель}
            - results: словарь с метриками моделей {имя: {'IoU':
..., 'threshold': ...}}
            - X_train, X_val, X_test: массивы данных (для
статистики)
            """
            os.makedirs(self.train_dir, exist_ok=True)

            for name, model in models.items():
                safe_name = name.replace(' ', '_').replace('(',
'').replace(')', '').replace('=', '')
                model_path = os.path.join(self.train_dir,
f'{safe_name}.keras')
                model.save(model_path)
                print(f"Модель сохранена: {model_path}")

            results_path = os.path.join(self.train_dir,
'segmentation_results.pkl')
            with open(results_path, 'wb') as f:
                pickle.dump(results, f)
            print(f"Результаты сравнения сохранены:
{results_path}")

            best_model_name = max(results, key=lambda n:
results[n]['IoU'])
            best_iou = results[best_model_name]['IoU']

            metadata = {
```

```
                'train_samples': len(X_train),
                'val_samples': len(X_val),
                'test_samples': len(X_test),
                'best_iou': float(best_iou),
                'best_model': best_model_name,
                'best_threshold':
float(results[best_model_name].get('threshold', 0.5)),
                'tversky_params': {'alpha': 0.7, 'beta': 0.3},
                'date': pd.Timestamp.now().isoformat()
            }

            metadata_path      =      os.path.join(self.train_dir,
'pract34_metadata.json')
            with open(metadata_path, 'w', encoding='utf-8') as f:
                json.dump(metadata,     f,     ensure_ascii=False,
indent=2)
            print(f"Метаданные сохранены: {metadata_path}")

            print("\nВсе файлы сохранены в папке:", self.train_dir)
            print("  - все обученные модели (*.keras)")
            print("  - segmentation_results.pkl")
            print("  - pract34_metadata.json")
            print("      -   графики   (threshold_analysis_*.png,
test_predictions*.png)")
```

Файл water_quality_pipeline_extended.py (новые методы):

```
        def norm_band(self, band):
            """
            Функция нормализации каналов для визуализации (отсекаем
2% и 98% процентили)
            """
            p2, p98 = np.percentile(band, (2, 98))
            return np.clip((band - p2) / (p98 - p2 + 1e-10), 0, 1)
* 255
```

```
        def create_dataset_from_pract2(self,
raster_path='data/delta_volga.tif',
                                       output_dir='water_dataset',
patch_size=256, stride=32,
                                       save_4band=True,
                                       dynamic_world=True,
dynamic_world_path='data/dynamic_world.tif'):
            """
            Использует спутниковый снимок из Практической работы №2
            и создает набор патчей (изображений) с соответствующими
            масками воды.
            Параметры:
            - raster_path: путь к GeoTIFF снимку
            - output_dir: папка для сохранения датасета
            - patch_size: размер патча (обычно 256x256)
            - stride: шаг нарезки (меньше шаг = больше патчей)
            - save_4band: сохранять ли 4-канальные данные (RGB +
NIR)
            - dynamic_world: эталон по Dynamic World того же снимка
(иначе NDWI > 0.15)
            - dynamic_world_path: путь к маске Dynamic World
(download_dynamic_world.py)
            """
            print("\n" + "=" * 60)
            print("Задача 3.4: Создание датасет для сегментации")
            print("=" * 60)
            self.dataset_dir = output_dir

            os.makedirs(f'{output_dir}/images', exist_ok=True)
            os.makedirs(f'{output_dir}/masks', exist_ok=True)
            if save_4band:
                os.makedirs(f'{output_dir}/images_4band',
exist_ok=True)
```

```
            for sub in ['images', 'masks', 'images_4band']:
                if os.path.isdir(f'{output_dir}/{sub}'):
                    for fname in
os.listdir(f'{output_dir}/{sub}'):
                        if fname.startswith('patch_'):

os.remove(f'{output_dir}/{sub}/{fname}')

            try:
                with rasterio.open(raster_path) as src:
                    red = src.read(1).astype(np.float32)
                    green = src.read(2).astype(np.float32)
                    blue = src.read(3).astype(np.float32)
                    nir = src.read(4).astype(np.float32)

                    if dynamic_world:
                        water_mask = self.load_reference_mask(src,
dynamic_world_path)
                    else:
                        ndwi = (green - nir) / (green + nir + 1e-
10)
                        water_mask = (ndwi >
0.10).astype(np.uint8)

                    rgb = np.stack([self.norm_band(red),
self.norm_band(green),
                                    self.norm_band(blue)], axis=-
1).astype(np.uint8)

                    h, w = rgb.shape[:2]
                    patch_idx = 0
                    positions = []
                    for y in range(0, h - patch_size + 1, stride):
                        for x in range(0, w - patch_size + 1,
stride):
```

```
                            mask_patch = water_mask[y:y +
patch_size, x:x + patch_size]

                            if np.sum(mask_patch) > patch_size **
2 * 0.01:

                                Image.fromarray(rgb[y:y +
patch_size, x:x + patch_size]).save(

f'{output_dir}/images/patch_{patch_idx:04d}.png'
                                )

                                if save_4band:
                                    bands = np.stack([
                                        red[y:y + patch_size, x:x
+ patch_size],
                                        green[y:y + patch_size,
x:x + patch_size],
                                        blue[y:y + patch_size, x:x
+ patch_size],
                                        nir[y:y + patch_size, x:x
+ patch_size]
                                    ])


np.save(f'{output_dir}/images_4band/patch_{patch_idx:04d}.npy',
bands)

                                Image.fromarray((mask_patch *
255).astype(np.uint8)).save(

f'{output_dir}/masks/patch_{patch_idx:04d}.png'
                                )
```

```
                                positions.append({'name':
f'patch_{patch_idx:04d}.png',
                                                  'x': x, 'y': y,
'size': patch_size})
                                patch_idx += 1


pd.DataFrame(positions).to_csv(f'{output_dir}/patches.csv',
index=False)

                    print(f"Создано {patch_idx} патчей")
                    print(f"  -  images/:  RGB  изображения  для
визуализации")
                    print(f" - masks/: бинарные маски воды (1 -
вода, 0 - суша)")
                    print(f" - patches.csv: положение патчей на
снимке")

                    if save_4band:
                        print(f"  -  images_4band/:  4-канальные
данные для NDWI")

                    return patch_idx
            except Exception as e:
                print(f"Ошибка при создании датасета: {e}")
                return 0


        def  load_reference_mask(self,  src,  reference_path,
water_classes=(0,)):
            """
            Маска воды по карте классов Dynamic World (0 - вода).
            Маска должна быть на той же сетке, что и снимок
            """
            with rasterio.open(reference_path) as ref:
```

```
                if ref.crs != src.crs or ref.transform != src.transform or ref.shape != src.shape:
                    raise ValueError("Сетка маски не совпадает со снимком")
                classes = ref.read(1)
            return np.isin(classes, water_classes).astype(np.uint8)


        def visualize_dataset(self, n_samples=3, filename='dataset_samples.png'):
            """
            Визуализация случайных патчей датасета: RGB, маска воды и наложение маски на снимок
            """
            images_dir = f'{self.dataset_dir}/images'
            masks_dir = f'{self.dataset_dir}/masks'
            if not os.path.isdir(images_dir) or not os.path.isdir(masks_dir):
                print(f"Датасет не найден: {self.dataset_dir}")
                return

            files = sorted(f for f in os.listdir(images_dir) if f.endswith('.png'))
            if not files:
                print("В датасете нет патчей")
                return

            samples = random.sample(files, min(n_samples, len(files)))

            fig, axes = plt.subplots(len(samples), 3, figsize=(12, 4 * len(samples)), squeeze=False)
            for row, name in enumerate(samples):
```

```
                image                                                    =
np.array(Image.open(f'{images_dir}/{name}'))
                mask = np.array(Image.open(f'{masks_dir}/{name}'))
> 0

                overlay = image.copy()
                overlay[mask]  =  (0.5  *  overlay[mask]  +  0.5  *
np.array([0, 0, 255])).astype(np.uint8)

                axes[row, 0].imshow(image)
                axes[row, 0].set_title(f'{name}: RGB')
                axes[row, 1].imshow(mask, cmap='gray')
                axes[row,  1].set_title(f'Маска  воды  ({mask.mean()
* 100:.1f}% воды)')
                axes[row, 2].imshow(overlay)
                axes[row, 2].set_title('Наложение маски')
                for ax in axes[row]:
                    ax.axis('off')

            plt.tight_layout()
            plt.savefig(f'{self.dataset_dir}/{filename}', dpi=150)
            plt.show()


        def test_methods_on_patches(self, n_samples=3):
            """
            в                                                    class
WaterQualityPipelineExtended(WaterQualityPipeline):
            Тестирование методов сегментации на патчах
            Сравнивает различные пороговые методы сегментации:
            - Глобальный порог по NDWI
            - Метод Оцу
            - Адаптивный порог
            - NDWI с порогом 0.15
            """
```

```
            print("\n" + "=" * 60)
            print("Задача 1.2: Тестирование методов сегментации")
            print("=" * 60)
            image_dir = f'{self.dataset_dir}/images'
            mask_dir = f'{self.dataset_dir}/masks'
            npy_dir = f'{self.dataset_dir}/images_4band'
            if not all(os.path.exists(d) for d in [image_dir,
mask_dir, npy_dir]):
                print("Сначала создайте датасет с 4-канальными
данными")
                return
            files = sorted(set(os.listdir(image_dir)) &
set(os.listdir(mask_dir)))
            n_samples = min(n_samples, len(files))
            selected = random.sample(files, n_samples)
            selected.sort()
            images_rgb, true_masks, images_4band = [], [], []
            for fname in selected:

images_rgb.append(plt.imread(f'{image_dir}/{fname}'))
                mask = plt.imread(f'{mask_dir}/{fname}')
                true_masks.append(mask[:, :, 0] if mask.ndim == 3
else mask)
                npy_path = f'{npy_dir}/{fname[:-4]}.npy'
                if os.path.exists(npy_path):
                    images_4band.append(np.load(npy_path))
                else:
                    images_4band.append(np.zeros((256, 256, 4),
dtype=np.float32))
            method_results = {
                'Глобальный (0.5)':
[self.segmenter.global_threshold(img4) for img4 in images_4band],
                'Оцу': [self.segmenter.otsu_threshold(img4) for
img4 in images_4band],
```

```
                'Адаптивный':
[self.segmenter.adaptive_threshold(img4) for img4 in images_4band],
                'NDWI                                          (0.15)':
[self.segmenter.ndwi_threshold(img4) for img4 in images_4band]
            }
            postprocessed = {}
            for method_name, preds in method_results.items():
                post_preds                                          =
[self.segmenter.postprocess_mask(pred) for pred in preds]
                postprocessed[f'{method_name}+Пост'] = post_preds
            self._show_results(images_rgb,             true_masks,
method_results, 'segmentation_results.png')
            self._show_results(images_rgb,             true_masks,
postprocessed, 'segmentation_results_post.png')
            method_results.update(postprocessed)
            print("\n" + "=" * 60)
            print("Итоговые метрики (средний IoU по патчам)")
            print("=" * 60)
            for method_name in method_results.keys():
                ious = [

self.segmenter.calculate_metrics(true_masks[i],
method_results[method_name][i])['IoU']
                    for i in range(len(images_rgb))
                ]
                print(f"{method_name:25}: {np.mean(ious):.3f}")


        def  _show_results(self,  images,  true_masks,  methods,
filename='segmentation_results.png'):
            """
            в класс WaterQualityPipelineExtended
            Вспомогательный метод для визуализации результатов
            Создает сетку изображений: оригинал, идеальная маска и
            результаты всех методов.
```

```
            """
            n_samples = len(images)
            n_methods = len(methods)
            fig, axes = plt.subplots(n_samples, n_methods + 2,
figsize=(4 * (n_methods + 2), 4 * n_samples))
            if n_samples == 1:
                axes = axes.reshape(1, -1)
            print("\nIoU по методам (для первого патча):")
            print("-" * 50)
            for i in range(n_samples):
                axes[i, 0].imshow(images[i])
                axes[i, 0].set_title('RGB', fontsize=12)
                axes[i, 0].axis('off')
                axes[i,  1].imshow(true_masks[i],  cmap='Blues',
vmin=0, vmax=1)
                axes[i, 1].set_title('Ground Truth', fontsize=12)
                axes[i, 1].axis('off')
                for     j,     (method_name,     preds)     in
enumerate(methods.items()):
                    col = j + 2
                    pred = preds[i]
                    if len(pred.shape) == 3 and pred.shape[0] == 4:
                        pred = np.mean(pred, axis=0)
                    elif len(pred.shape) == 3:
                        pred = np.mean(pred, axis=-1)
                    if  len(pred.shape)  ==  2  and  pred.shape  ==
true_masks[i].shape:
                        iou                                    =
self.segmenter.calculate_metrics(true_masks[i], pred)['IoU']
                        axes[i,  col].imshow(pred,  cmap='Blues',
vmin=0, vmax=1)
                        axes[i,
col].set_title(f'{method_name}\nIoU: {iou:.3f}', fontsize=10)
                        if i == 0:
                            print(f"{method_name:25}: {iou:.3f}")
```

```
                    else:
                        axes[i,
col].imshow(np.zeros_like(true_masks[i]), cmap='Blues')
                        axes[i,
col].set_title(f'{method_name}\nОшибка', fontsize=10)
                    axes[i, col].axis('off')
            plt.tight_layout()
            plt.savefig(f'{self.dataset_dir}/{filename}', dpi=150,
bbox_inches='tight')
            plt.show()


        def     plot_confusion_matrix(self,     patch_name=None,
target_water=0.2, filename='confusion_matrix.png'):
            """
            Матрица ошибок в пиксельном пространстве для одного
патча и всех пороговых методов.
            Карта ошибок: зеленый - TP (вода найдена), красный - FP
(ложная вода),
            желтый - FN (пропущенная вода), серый - TN (фон верно)
            patch_name=None - берется патч с долей воды, ближайшей
к target_water
            """
            print("\n" + "=" * 60)
            print("Задача 2.2: Матрица ошибок для патча")
            print("=" * 60)
            image_dir = f'{self.dataset_dir}/images'
            mask_dir = f'{self.dataset_dir}/masks'
            npy_dir = f'{self.dataset_dir}/images_4band'

            def load_mask(name):
                mask = plt.imread(f'{mask_dir}/{name}')
                return (mask[:, :, 0] if mask.ndim == 3 else mask)
> 0.5
```

```
            if patch_name is None:
                files = sorted(set(os.listdir(image_dir)) &
set(os.listdir(mask_dir)))
                patch_name = min(files, key=lambda f:
abs(load_mask(f).mean() - target_water))

            image = plt.imread(f'{image_dir}/{patch_name}')
            true_mask = load_mask(patch_name)
            image_4band = np.load(f'{npy_dir}/{patch_name[:-
4]}.npy')
            print(f"Патч {patch_name}, вода {true_mask.mean() *
100:.1f}% пикселей")

            methods = {
                'Глобальный (0.5)':
self.segmenter.global_threshold(image_4band),
                'Оцу': self.segmenter.otsu_threshold(image_4band),
                'Адаптивный':
self.segmenter.adaptive_threshold(image_4band),
                'NDWI (0.15)':
self.segmenter.ndwi_threshold(image_4band),
            }

            fig, axes = plt.subplots(1, 2, figsize=(8, 4))
            axes[0].imshow(image)
            axes[0].set_title(f'RGB: {patch_name}')
            axes[1].imshow(true_mask, cmap='Blues', vmin=0,
vmax=1)
            axes[1].set_title(f'Эталон ({true_mask.mean() *
100:.1f}% воды)')
            for ax in axes:
                ax.axis('off')
            plt.tight_layout()
            plt.savefig(f'{self.dataset_dir}/{filename[:-
4]}_patch.png', dpi=150, bbox_inches='tight')
```

```
            plt.show()

            colors = {'TP': (0, 0.8, 0), 'FP': (0.9, 0, 0), 'FN':
(1, 0.85, 0), 'TN': (0.6, 0.6, 0.6)}
            fig, axes = plt.subplots(2, len(methods), figsize=(4 *
len(methods), 8))
            fig.legend(handles=[Patch(color=c, label=k) for k, c in
colors.items()],
                       loc='lower  center',  bbox_to_anchor=(0.5,
1.0), ncol=4, fontsize=12, title='Карта ошибок')

            rows = []
            n = true_mask.size
            for col, (name, pred) in enumerate(methods.items()):
                pred = pred > 0.5
                tp, fp = (true_mask & pred).sum(), (~true_mask &
pred).sum()
                fn, tn = (true_mask & ~pred).sum(), (~true_mask &
~pred).sum()

                error_map = np.zeros((*true_mask.shape, 3))
                for key, area in [('TP', true_mask & pred), ('FP',
~true_mask & pred),
                                  ('FN',  true_mask  &  ~pred),
('TN', ~true_mask & ~pred)]:
                    error_map[area] = colors[key]
                iou = tp / (tp + fp + fn + 1e-10)
                axes[0, col].imshow(error_map)
                axes[0, col].set_title(f'{name}\nIoU = {iou:.3f}')
                axes[0, col].axis('off')

                matrix = np.array([[tp, fn], [fp, tn]])
                axes[1,  col].imshow(matrix  /  n,  cmap='Blues',
vmin=0, vmax=1)
                for (r, c), value in np.ndenumerate(matrix):
```

```
                    label = [['TP', 'FN'], ['FP', 'TN']][r][c]
                    axes[1,            col].text(c,            r,
f'{label}\n{value}\n({value   /   n   *   100:.1f}%)',   ha='center',
va='center',
                                        color='white' if value / n >
0.5 else 'black', fontsize=11)
                axes[1, col].set_xticks([0, 1], ['Вода', 'Фон'])
                axes[1, col].set_yticks([0, 1], ['Вода', 'Фон'])
                axes[1, col].set_xlabel('Предсказание')
                axes[1, col].set_ylabel('Эталон')

                rows.append({'Метод': name, 'TP': tp, 'FP': fp,
'FN': fn, 'TN': tn, 'IoU': iou,
                             'Precision': tp / (tp + fp + 1e-10),
'Recall': tp / (tp + fn + 1e-10)})

            plt.tight_layout()
            plt.savefig(f'{self.dataset_dir}/{filename}', dpi=150,
bbox_inches='tight')
            plt.show()

            table = pd.DataFrame(rows).set_index('Метод')
            print(table.round(3).to_string())
            return table


        def     load_dataset(self,     data_dir='water_dataset',
img_size=(256, 256), train_size=0.7, val_size=0.15,
                         block_size=448):
            """
            Пространственно разделяет данные на три части по
клеткам снимка,
            случайно распределенным между выборками:
            - Train (обучение) - около 70%
            - Val (валидация) - около 15%
```

```
            - Test (тест) - около 15%
            Патчи, пересекающие границы клеток разных выборок,
исключаются, чтобы
            перекрывающиеся патчи не попадали в разные выборки
            """
            print("\n" + "=" * 60)
            print("Загрузка и разделение датасета")
            print("=" * 60)

            image_dir = f'{data_dir}/images'
            mask_dir = f'{data_dir}/masks'
            positions_path = f'{data_dir}/patches.csv'

            if      not      (os.path.exists(image_dir)      and
os.path.exists(mask_dir)):
                print("Сначала создайте датасет")
                return None
            if not os.path.exists(positions_path):
                print("Нет файла patches.csv с положением патчей,
пересоздайте датасет")
                return None

            common_files    =    set(os.listdir(image_dir))    &
set(os.listdir(mask_dir))
            positions = pd.read_csv(positions_path)
            positions                                          =
positions[positions['name'].isin(common_files)].reset_index(drop=T
rue)
            print(f"Найдено  {len(positions)}  пар  изображение-
маска")

            images, masks = [], []
            for img_file in positions['name']:
                mask_file = img_file
```

```
                img = plt.imread(f'{image_dir}/{img_file}')
                if img.max() > 1.0:
                    if img.max() <= 255:
                        img = img.astype(np.float32) / 255.0
                    else:
                        p2, p98 = np.percentile(img, (2, 98))
                        img = np.clip((img - p2) / (p98 - p2 + 1e-
8), 0, 1)

                if img.shape[:2] != img_size:
                    img       =       resize(img,       img_size,
preserve_range=True, anti_aliasing=True)

                mask = plt.imread(f'{mask_dir}/{mask_file}')
                if mask.ndim == 3:
                    mask = mask[:, :, 0]
                if mask.shape[:2] != img_size:
                    mask      =      resize(mask,      img_size,
preserve_range=True, anti_aliasing=False)
                mask = (mask > 0.5).astype(np.float32)

                images.append(img.astype(np.float32))
                masks.append(mask)

            X, y = np.array(images), np.array(masks)
            print(f"Размерность данных: X {X.shape}, y {y.shape}")

            split  =  self.block_split(positions['x'].to_numpy(),
positions['y'].to_numpy(),
                                       y.mean(axis=(1,          2)),
int(positions['size'].iloc[0]),
                                       block_size,      train_size,
val_size)
            if split is None:
```

```
                print("Не удалось разделить датасет: слишком мало
патчей")
                return None
            (train_idx, val_idx, test_idx), cell_set = split

            X_train, y_train = X[train_idx], y[train_idx]
            X_val, y_val = X[val_idx], y[val_idx]
            X_test, y_test = X[test_idx], y[test_idx]

            n_used = len(train_idx) + len(val_idx) + len(test_idx)
            n_cells = [list(cell_set.values()).count(k) for k in
range(3)]
            print(f"\nКлетки {block_size}x{block_size}: Train
{n_cells[0]}, Val {n_cells[1]}, Test {n_cells[2]}")
            print(f"Исключено патчей на границах клеток разных
выборок: {len(X) - n_used}")
            print(f"\nРазделение: Train {X_train.shape[0]}, Val
{X_val.shape[0]}, Test {X_test.shape[0]}")
            for name, part in [('Train', y_train), ('Val', y_val),
('Test', y_test)]:
                print(f"    {name:5}: {len(part) / n_used *
100:4.1f}% патчей, вода {part.mean() * 100:.1f}% пикселей")

            self.dataset = {
                'X_train': X_train, 'y_train': y_train,
                'X_val': X_val, 'y_val': y_val,
                'X_test': X_test, 'y_test': y_test
            }
            self.split_info = {
                'positions': positions, 'block_size': block_size,
'cell_set': cell_set,
                'train_idx': train_idx, 'val_idx': val_idx,
'test_idx': test_idx
            }
```

```
        return (X_train, y_train), (X_val, y_val), (X_test,
y_test)


    def                                plot_dataset_split(self,
raster_path='data/delta_volga.tif', filename='dataset_split.png'):
        """
        Визуализация разбиения датасета на снимке: клетки
        закрашены цветом своей выборки, точки - центры патчей
        (цвет - выборка, серый крестик - патч исключен на
границе клеток разных выборок)
        """
        if not hasattr(self, 'split_info'):
            print("Сначала выполните load_dataset()")
            return

        info = self.split_info
        positions, block_size, cell_set = info['positions'],
info['block_size'], info['cell_set']
        patch_size = int(positions['size'].iloc[0])

        with rasterio.open(raster_path) as src:
            rgb                                                =
np.stack([self.norm_band(src.read(i).astype(np.float32)) for i in
(1, 2, 3)],
                                 axis=-1).astype(np.uint8)
        h, w = rgb.shape[:2]

        names = ['Train', 'Val', 'Test']
        colors = ['tab:green', 'tab:orange', 'tab:purple']

        fig, ax = plt.subplots(figsize=(10, 11))
        ax.imshow(rgb)

        for (r, c), part in cell_set.items():
```

```
                ax.add_patch(Rectangle((c * block_size, r *
block_size), block_size, block_size,
                                       facecolor=colors[part],
edgecolor='white',
                                       alpha=0.3, linewidth=1))

            centers_x = positions['x'].to_numpy() + patch_size / 2
            centers_y = positions['y'].to_numpy() + patch_size / 2
            used = np.zeros(len(positions), dtype=bool)
            for name, color, key in zip(names, colors,
['train_idx', 'val_idx', 'test_idx']):
                idx = info[key]
                used[idx] = True
                ax.scatter(centers_x[idx], centers_y[idx], s=25,
color=color,
                           edgecolor='black', linewidth=0.5,
label=f'{name}: {len(idx)} патчей')
            ax.scatter(centers_x[~used], centers_y[~used], s=20,
marker='x', color='lightgray',
                       label=f'Исключены: {(~used).sum()} патчей')

            ax.set_xlim(0, w)
            ax.set_ylim(h, 0)
            ax.set_title(f'Разбиение датасета: клетки
{block_size}x{block_size} пикселей')
            ax.legend(loc='upper left', bbox_to_anchor=(1.01, 1))
            ax.axis('off')

            plt.tight_layout()
            plt.savefig(self.output_dir + filename, dpi=150,
bbox_inches='tight')
            plt.show()
```

```
        def block_split(self, xs, ys, water, patch_size,
block_size=448,
                        train_size=0.7,            val_size=0.15,
n_trials=200, min_patches=10, seed=42):
            """
            Разделение по клеткам: снимок делится на клетки
block_size x block_size,
            клетки случайно распределяются между train/val/test в
заданных долях.
            Из n_trials случайных вариантов выбирается тот, где
доли патчей ближе
            к заданным, а доля воды в выборках - одинаковая.
            Патч попадает в выборку, только если все клетки под ним
относятся к ней
            """
            rng = np.random.default_rng(seed)
            target = np.array([train_size, val_size, 1 - train_size
- val_size])
            cells = [{(r, c) for r in range(y // block_size, (y +
patch_size - 1) // block_size + 1)
                             for c in range(x // block_size, (x +
patch_size - 1) // block_size + 1)}
                     for x, y in zip(xs, ys)]
            all_cells = sorted(set().union(*cells))
            n_val = round(len(all_cells) * target[1])
            n_test = round(len(all_cells) * target[2])
            best = None
            for _ in range(n_trials):
                order = rng.permutation(len(all_cells))
                cell_set = {all_cells[j]: 1 if i < n_val else 2 if
i < n_val + n_test else 0
                            for i, j in enumerate(order)}
                labels = np.full(len(xs), -1)
                for i, patch_cells in enumerate(cells):
                    sets = {cell_set[c] for c in patch_cells}
```

```
                    if len(sets) == 1:
                        labels[i] = sets.pop()
                sizes = np.array([(labels == k).sum() for k in
range(3)])
                if (sizes < min_patches).any():
                    continue
                water_share = np.array([water[labels == k].mean()
for k in range(3)])
                score = (np.abs(sizes / sizes.sum() - target).sum()
                         + 0.5 * np.abs(water_share -
water_share.mean()).sum() / water_share.mean()
                         + 0.5 * (labels == -1).mean())
                if best is None or score < best[0]:
                    best = (score, labels, cell_set)
            if best is None:
                return None
            labels = best[1]
            return tuple(np.where(labels == k)[0] for k in
range(3)), best[2]


        def get_augmentation(self, p=0.5):
            """
            Аугментация позволяет искусственно увеличить размер
            датасета за счет случайных преобразований изображений.
            """
            return A.Compose([
                A.RandomRotate90(p=0.5),
                A.HorizontalFlip(p=0.5),
                A.VerticalFlip(p=0.5),
                A.Transpose(p=0.3),
                A.Affine(translate_percent=(-0.1, 0.1),
scale=(0.9, 1.1), rotate=(-30, 30), p=0.3),
```

```
                A.RandomBrightnessContrast(brightness_limit=0.2,
contrast_limit=0.2, p=0.5),
                A.ColorJitter(brightness=0.2,        contrast=0.2,
saturation=0.2, hue=0.1, p=0.4),
                A.HueSaturationValue(hue_shift_limit=15,
sat_shift_limit=25, val_shift_limit=15, p=0.3),

                A.GaussNoise(std_range=(0.012, 0.025), p=0.3),
                A.ISONoise(color_shift=(0.01, 0.05), p=0.2),
                A.GaussianBlur(blur_limit=(3, 5), p=0.3),
                A.MedianBlur(blur_limit=5, p=0.2),

                A.RandomFog(fog_coef_range=(0.1, 0.3), p=0.2),
                A.RandomRain(slant_range=(-10, 10), drop_width=1,
p=0.1),
                A.RandomSnow(snow_point_range=(0.1, 0.3), p=0.1),

                A.CoarseDropout(num_holes_range=(8,            8),
hole_height_range=(32, 32),
                                hole_width_range=(32, 32), fill=0,
p=0.2),
            ], p=p)


        def     apply_augmentation(self,     images,     masks,
augmentor=None):
            """
            Применение аугментации к батчу изображений
            Преобразует изображения и соответствующие маски,
сохраняя
            их соответствие.
            """
            if augmentor is None:
                augmentor = self.get_augmentation()
```

```
            aug_images, aug_masks = [], []
            for i in range(len(images)):
                img_uint8 = (images[i] * 255).astype(np.uint8) if
images[i].max() <= 1.0 else images[i].astype(np.uint8)
                mask_uint8 = (masks[i] * 255).astype(np.uint8)

                augmented       =       augmentor(image=img_uint8,
mask=mask_uint8)


aug_images.append(augmented['image'].astype(np.float32) / 255.0)

aug_masks.append(augmented['mask'].astype(np.float32) / 255.0)

            return np.array(aug_images), np.array(aug_masks)


        def visualize_augmentation(self, n_samples=4):
            """
            Демонстрация аугментации на примерах
            Показывает   оригиналы   и   аугментированные   версии
изображений
            и масок.
            """
            print("\n" + "=" * 60)
            print("Демонстрация аугментации")
            print("=" * 60)

            if not hasattr(self, 'dataset') or 'X_train' not in
self.dataset:
                print("Сначала выполните load_dataset()")
                return

            sample_images = self.dataset['X_train'][:n_samples]
            sample_masks = self.dataset['y_train'][:n_samples]
```

```
            aug_images,                 aug_masks                 =
self.apply_augmentation(sample_images, sample_masks)

            fig, axes = plt.subplots(2, n_samples * 2, figsize=(4
* n_samples * 2, 8))
            for i in range(n_samples):
                axes[0, i * 2].imshow(sample_images[i])
                axes[0, i * 2].set_title(f'Оригинал {i + 1}')
                axes[0, i * 2].axis('off')

                axes[0,   i   *   2   +   1].imshow(sample_masks[i],
cmap='Blues', vmin=0, vmax=1)
                axes[0, i * 2 + 1].set_title(f'Маска {i + 1}')
                axes[0, i * 2 + 1].axis('off')

                axes[1, i * 2].imshow(aug_images[i])
                axes[1, i * 2].set_title(f'Аугмент. {i + 1}')
                axes[1, i * 2].axis('off')

                axes[1,   i   *   2   +   1].imshow(aug_masks[i],
cmap='Blues', vmin=0, vmax=1)
                axes[1, i * 2 + 1].set_title(f'Маска аугм.')
                axes[1, i * 2 + 1].axis('off')

            plt.tight_layout()

plt.savefig(f'{self.dataset_dir}/augmentation_demo.png',   dpi=150,
bbox_inches='tight')
            plt.show()


        def  find_optimal_threshold(self,  model,  X_val,  y_val,
model_name="", thresholds=np.arange(0.3, 0.9, 0.05)):
            """
```

```
            Поиск оптимального порога бинаризации для модели
            Перебирает пороги от 0.3 до 0.85 и выбирает тот,
            при котором IoU на валидационной выборке максимален.
            """
            print(f"\nПоиск оптимального порога для
{model_name}...")

            best_threshold, best_iou = 0.5, 0
            threshold_results = []

            n_samples = min(30, len(X_val))
            X_sample, y_sample = X_val[:n_samples],
y_val[:n_samples]
            preds = model.predict(X_sample, batch_size=4,
verbose=0)[..., 0]

            for threshold in thresholds:
                ious = []
                for i in range(n_samples):
                    pred_bin =
self.segmenter.postprocess_mask((preds[i] >
threshold).astype(np.float32))
                    iou =
self.segmenter.calculate_metrics(y_sample[i], pred_bin)['IoU']
                    ious.append(iou)

                mean_iou = np.mean(ious)
                threshold_results.append((threshold, mean_iou))

                if mean_iou > best_iou:
                    best_iou, best_threshold = mean_iou, threshold
                    print(f"Порог {threshold:.2f}: IoU =
{mean_iou:.4f} (НОВЫЙ ЛУЧШИЙ)")
                else:
```

```
                    print(f"Порог {threshold:.2f}: IoU = {mean_iou:.4f}")

            self.plot_threshold_analysis(threshold_results, model_name)

            return best_threshold, best_iou


        def plot_threshold_analysis(self, threshold_results, model_name):
            """
            ВИЗУАЛИЗАЦИЯ ЗАВИСИМОСТИ IoU ОТ ПОРОГА
            Показывает, как меняется качество сегментации при изменении
            порога, и отмечает оптимальное значение.
            """
            thresholds, ious = zip(*threshold_results)

            plt.figure(figsize=(10, 6))
            plt.plot(thresholds, ious, 'b-o', linewidth=2, markersize=8)
            plt.xlabel('Порог бинаризации')
            plt.ylabel('IoU')
            plt.title(f'Зависимость качества от порога\n{model_name}')
            plt.grid(True, alpha=0.3)

            best_idx = np.argmax(ious)
            best_threshold, best_iou = thresholds[best_idx], ious[best_idx]

            plt.axvline(x=best_threshold, color='red', linestyle='--', alpha=0.7)
```

```
            plt.scatter([best_threshold], [best_iou], color='red',
s=200, zorder=5)
            plt.annotate(
                f'Оптимум:
{best_threshold:.2f}\nIoU={best_iou:.4f}',
                xy=(best_threshold, best_iou), xytext=(10, -35),
textcoords='offset points',
                fontsize=10, color='red'
            )

            plt.tight_layout()
            plt.savefig(

f'{self.train_dir}/threshold_analysis_{model_name.replace("      ",
"_")}.png',
                dpi=150, bbox_inches='tight'
            )
            plt.show()


        def visualize_predictions(self, models_dict, X_test,
y_test, n_samples=4, save=True):
            """
            Визуализация предсказаний моделей с адаптивными
порогами
            Показывает для каждого тестового примера:
            - оригинал
            - идеальную маску
            - предсказания всех моделей
            - карту ошибок лучшей модели
            """
            print("\n" + "=" * 60)
            print("Визуализация предсказаний")
            print("=" * 60)
```

```
            indices = np.random.choice(len(X_test), min(n_samples,
len(X_test)), replace=False)
            n_models = len(models_dict)

            fig, axes = plt.subplots(len(indices), n_models + 3,
figsize=(4 * (n_models + 3), 4 * len(indices)))
            if len(indices) == 1:
                axes = axes.reshape(1, -1)

            optimal_thresholds = {'Dice Loss': 0.65, 'Tversky
Loss': 0.5, 'Combined Loss': 0.45}

            for row, idx in enumerate(indices):
                img, true_mask = X_test[idx],
y_test[idx].squeeze()
                col = 0

                axes[row, col].imshow(img)
                axes[row, col].set_title('Снимок')
                axes[row, col].axis('off')
                col += 1

                axes[row, col].imshow(true_mask, cmap='Blues',
vmin=0, vmax=1)
                axes[row, col].set_title('Идеальная маска')
                axes[row, col].axis('off')
                col += 1

                ious = []
                for name, model in models_dict.items():
                    pred_raw = model.predict(img[np.newaxis],
verbose=0)[0, :, :, 0]
                    threshold = optimal_thresholds.get(name, 0.5)
```

```
                    if name == 'Dice Loss' and pred_raw.mean() >
0.7:
                        try:
                            pred_norm      =      ((pred_raw      -
pred_raw.min())  /  (pred_raw.max()  -  pred_raw.min()  +  1e-8)  *
255).astype(np.uint8)
                            threshold  =  threshold_otsu(pred_norm)
/ 255.0
                        except:
                            threshold = 0.7

                    pred_bin                                        =
self.segmenter.postprocess_mask((pred_raw                           >
threshold).astype(np.float32))
                    iou                                             =
self.segmenter.calculate_metrics(true_mask, pred_bin)['IoU']
                    ious.append((name, iou))

                    axes[row,  col].imshow(pred_bin,  cmap='Blues',
vmin=0, vmax=1)
                    axes[row,
col].set_title(f'{name}\nпорог={threshold:.2f}\nIoU={iou:.3f}',
fontsize=9)
                    axes[row, col].axis('off')
                    col += 1

                if ious:
                    best_name, best_iou = max(ious, key=lambda x:
x[1])
                    best_model = models_dict[best_name]
                    pred_raw = best_model.predict(img[np.newaxis],
verbose=0)[0, :, :, 0]
                    threshold  =  optimal_thresholds.get(best_name,
0.5)
```

```
                    pred_bin =
self.segmenter.postprocess_mask((pred_raw >
threshold).astype(np.float32))

                    error_map = np.zeros((*pred_bin.shape, 3))
                    tp_mask = (true_mask > 0.5) & (pred_bin > 0.5)
                    fp_mask = (true_mask <= 0.5) & (pred_bin > 0.5)
                    fn_mask = (true_mask > 0.5) & (pred_bin <= 0.5)
                    error_map[tp_mask] = [0, 1, 0]
                    error_map[fp_mask] = [1, 0, 0]
                    error_map[fn_mask] = [1, 1, 0]

                    axes[row, -1].imshow(error_map)
                    axes[row, -1].set_title(f'Ошибки\n{best_name}:
{best_iou:.3f}', fontsize=10)
                    axes[row, -1].axis('off')

            plt.tight_layout()
            if save:

plt.savefig(f'{self.dataset_dir}/test_predictions.png',    dpi=150,
bbox_inches='tight')
            plt.show()


        def            visualize_predictions_with_thresholds(self,
models_dict, X_test, y_test, optimal_thresholds=None, n_samples=4,
save=True):
            """
            Визуализация предсказаний с оптимальными порогами
            Использует найденные ранее оптимальные пороги для
каждой модели.
            """
            print("\n" + "=" * 60)
            print("Визуализация - оптимальные пороги")
```

```
            print("=" * 60)

            if optimal_thresholds is None:
                optimal_thresholds = {name: 0.5 for name in
models_dict.keys()}

            indices = np.random.choice(len(X_test), min(n_samples,
len(X_test)), replace=False)
            n_models = len(models_dict)

            fig, axes = plt.subplots(len(indices), n_models + 3,
figsize=(4 * (n_models + 3), 4 * len(indices)))
            if len(indices) == 1:
                axes = axes.reshape(1, -1)

            for row, idx in enumerate(indices):
                img, true_mask = X_test[idx],
y_test[idx].squeeze()
                col = 0

                axes[row, col].imshow(img)
                axes[row, col].set_title('Снимок')
                axes[row, col].axis('off')
                col += 1

                axes[row, col].imshow(true_mask, cmap='Blues',
vmin=0, vmax=1)
                axes[row, col].set_title('Идеальная маска')
                axes[row, col].axis('off')
                col += 1

                ious = []
                for name, model in models_dict.items():
                    pred_raw = model.predict(img[np.newaxis],
verbose=0)[0, :, :, 0]
```

```
                    threshold = optimal_thresholds.get(name, 0.5)
                    pred_bin                                        =
self.segmenter.postprocess_mask((pred_raw                           >
threshold).astype(np.float32))
                    iou                                             =
self.segmenter.calculate_metrics(true_mask, pred_bin)['IoU']
                    ious.append((name, iou))

                    axes[row, col].imshow(pred_bin, cmap='Blues',
vmin=0, vmax=1)
                    axes[row,
col].set_title(f'{name}\nпорог={threshold:.2f}\nIoU={iou:.3f}',
fontsize=9)
                    axes[row, col].axis('off')
                    col += 1

                if ious:
                    best_name, best_iou = max(ious, key=lambda x:
x[1])
                    best_model = models_dict[best_name]
                    pred_raw = best_model.predict(img[np.newaxis],
verbose=0)[0, :, :, 0]
                    threshold = optimal_thresholds.get(best_name,
0.5)
                    pred_bin                                        =
self.segmenter.postprocess_mask((pred_raw                           >
threshold).astype(np.float32))

                    error_map = np.zeros((*pred_bin.shape, 3))
                    tp_mask = (true_mask > 0.5) & (pred_bin > 0.5)
                    fp_mask = (true_mask <= 0.5) & (pred_bin > 0.5)
                    fn_mask = (true_mask > 0.5) & (pred_bin <= 0.5)
                    error_map[tp_mask] = [0, 1, 0]
                    error_map[fp_mask] = [1, 0, 0]
                    error_map[fn_mask] = [1, 1, 0]
```

```
                    axes[row, -1].imshow(error_map)
                    axes[row, -1].set_title(f'Ошибки\n{best_name}:
{best_iou:.3f}', fontsize=10)
                    axes[row, -1].axis('off')

            plt.tight_layout()
            if save:

plt.savefig(f'{self.dataset_dir}/test_predictions_optimal.png',
dpi=150, bbox_inches='tight')
            plt.show()
```

Файл water_segmetation.py:

```
import numpy as np
from skimage.exposure import equalize_hist
from skimage.filters import threshold_local, threshold_otsu
from     skimage.morphology     import     closing,     disk,
remove_small_objects


class WaterSegmentation():
    def __init__(self):
        self.results = {}


    def _get_ndwi(self, image):
        """
        Универсальное вычисление NDWI из любых данных
        NDWI = (Green - NIR) / (Green + NIR) для 4-канальных
        данных
        или (Green - Blue) / (Green + Blue) для RGB
        """
        if hasattr(image, 'shape') and len(image.shape) == 3:
            if image.shape[-1] == 4:
```

```
                    green, nir = image[:, :, 1].astype(np.float32),
image[:, :, 3].astype(np.float32)
                    return (green - nir) / (green + nir + 1e-8)
                elif image.shape[0] == 4:
                    green,   nir   =   image[1].astype(np.float32),
image[3].astype(np.float32)
                    return (green - nir) / (green + nir + 1e-8)
                elif image.shape[-1] >= 3:
                    green,      blue      =      image[:,      :,
1].astype(np.float32), image[:, :, 2].astype(np.float32)
                    return (green - blue) / (green + blue + 1e-8)
                return None


        def global_threshold(self, image, threshold=0.5):
            """
            Глобальный порог по NDWI
            """
            ndwi = self._get_ndwi(image)
            return (ndwi > threshold).astype(np.float32) if ndwi is
not None else np.zeros(image.shape[:2])


        def otsu_threshold(self, image):
            """
            Метод ОЦУ на NDWI
            Предварительно улучшает контраст (equalize_hist)
            и автоматически находит оптимальный порог
            """
            ndwi = self._get_ndwi(image)
            if ndwi is None:
                return np.zeros(image.shape[:2])

            ndwi_pos = np.clip(ndwi, 0, None)
            if ndwi_pos.max() == 0:
```

```
                return np.zeros(ndwi.shape)

            ndwi_eq = equalize_hist(ndwi_pos)
            try:
                return                  (ndwi_eq                  >
threshold_otsu(ndwi_eq)).astype(np.float32)
            except:
                return (ndwi > np.median(ndwi)).astype(np.float32)


        def   adaptive_threshold(self,   image,   block_size=101,
offset=-0.2):
            """
            Адаптивный порог по NDWI: свой порог для каждой области
изображения
            block_size - размер окна (нечетный), offset - смещение
порога
            """
            ndwi = self._get_ndwi(image)
            if ndwi is None:
                return np.zeros(image.shape[:2])

            local_thresh   =   threshold_local(ndwi,   block_size,
offset=offset)
            return (ndwi > local_thresh).astype(np.float32)


        def ndwi_threshold(self, image_4band, threshold=0.15):
            """
            NDWI порог для 4-канальных данных
            Без канала NIR использует grayscale: вода темнее суши,
порог - метод Оцу
            """
            if    image_4band.ndim    ==    3    and    4    in
(image_4band.shape[0], image_4band.shape[-1]):
```

```
                ndwi = self._get_ndwi(image_4band)
                return (ndwi > threshold).astype(np.float32)

            gray = image_4band.mean(axis=-1) if image_4band.ndim ==
3 else image_4band
            gray = gray.astype(np.float32)

            if gray.max() == gray.min():
                return np.zeros(gray.shape, dtype=np.float32)

            return                    (gray                    <
threshold_otsu(gray)).astype(np.float32)


        def postprocess_mask(self, mask, min_size=50):
            """
            в class WaterSegmentation
            Постобработка бинарной маски
            Удаляет маленькие объекты и заполняет дыры
            """
            mask_bool = mask > 0.5
            mask_clean      =      remove_small_objects(mask_bool,
min_size=min_size)
            return closing(mask_clean, disk(2)).astype(np.float32)


        def calculate_metrics(self, true_mask, pred_mask):
            """
            в class WaterSegmentation
            Расчет метрик качества сегментации
            IoU (Intersection over Union) - основная метрика
            """
            true_bin, pred_bin = true_mask > 0.5, pred_mask > 0.3
            tp = np.sum(true_bin & pred_bin)
            fp = np.sum(~true_bin & pred_bin)
```

```
            fn = np.sum(true_bin & ~pred_bin)
            return {'IoU': tp / (tp + fp + fn + 1e-10), 'TP': tp,
'FP': fp, 'FN': fn}
```

Файл unet-water.ipynb:

```
    import tensorflow as tf
    import pandas as pd
    from tensorflow.keras import layers, Model
    from   tensorflow.keras.callbacks   import   ModelCheckpoint,
EarlyStopping, ReduceLROnPlateau
    from skimage.transform import resize
    import albumentations as A
    import matplotlib.pyplot as plt
    import numpy as np
    import itertools
    import os

    def block_split(xs, ys, water, patch_size, block_size=448,
                        train_size=0.7,             val_size=0.15,
n_trials=200, min_patches=10, seed=42):
        rng = np.random.default_rng(seed)
        target = np.array([train_size, val_size, 1 - train_size -
val_size])

        cells = [{(r, c) for r in range(y // block_size, (y +
patch_size - 1) // block_size + 1)
                         for c in range(x // block_size, (x +
patch_size - 1) // block_size + 1)}
                 for x, y in zip(xs, ys)]
        all_cells = sorted(set().union(*cells))
        n_val = round(len(all_cells) * target[1])
        n_test = round(len(all_cells) * target[2])
        best = None
        for _ in range(n_trials):
```

```
            order = rng.permutation(len(all_cells))
            cell_set = {all_cells[j]: 1 if i < n_val else 2 if i <
n_val + n_test else 0
                        for i, j in enumerate(order)}
            labels = np.full(len(xs), -1)
            for i, patch_cells in enumerate(cells):
                sets = {cell_set[c] for c in patch_cells}
                if len(sets) == 1:
                    labels[i] = sets.pop()
            sizes = np.array([(labels == k).sum() for k in
range(3)])
            if (sizes < min_patches).any():
                continue
            water_share = np.array([water[labels == k].mean() for
k in range(3)])
            score = (np.abs(sizes / sizes.sum() - target).sum()
                     + 0.5 * np.abs(water_share -
water_share.mean()).sum() / water_share.mean()
                     + 0.5 * (labels == -1).mean())
            if best is None or score < best[0]:
                best = (score, labels, cell_set)
        if best is None:
            return None
        labels = best[1]
        return tuple(np.where(labels == k)[0] for k in range(3)),
best[2]


    def load_dataset(data_dir='water_dataset', img_size=(256,
256), train_size=0.7, val_size=0.15,
                     block_size=448):
        """
        Пространственно разделяет данные на три части по клеткам
снимка,
        случайно распределенным между выборками:
```

```
        - Train (обучение) - около 70%
        - Val (валидация) - около 15%
        - Test (тест) - около 15%
        Патчи, пересекающие границы клеток разных выборок, исключаются, чтобы
        перекрывающиеся патчи не попадали в разные выборки
        """
        print("\n" + "=" * 60)
        print("Загрузка и разделение датасета")
        print("=" * 60)

        image_dir = f'{data_dir}/images'
        mask_dir = f'{data_dir}/masks'
        positions_path = f'{data_dir}/patches.csv'

        if not (os.path.exists(image_dir) and os.path.exists(mask_dir)):
            print("Сначала создайте датасет")
            return None
        if not os.path.exists(positions_path):
            print("Нет файла patches.csv с положением патчей, пересоздайте датасет")
            return None

        common_files = set(os.listdir(image_dir)) & set(os.listdir(mask_dir))
        positions = pd.read_csv(positions_path)
        positions = positions[positions['name'].isin(common_files)].reset_index(drop=True)
        print(f"Найдено {len(positions)} пар изображение-маска")

        images, masks = [], []
        for img_file in positions['name']:
            mask_file = img_file
```

```
            img = plt.imread(f'{image_dir}/{img_file}')
            if img.max() > 1.0:
                if img.max() <= 255:
                    img = img.astype(np.float32) / 255.0
                else:
                    p2, p98 = np.percentile(img, (2, 98))
                    img = np.clip((img - p2) / (p98 - p2 + 1e-8),
0, 1)

            if img.shape[:2] != img_size:
                img = resize(img, img_size, preserve_range=True,
anti_aliasing=True)

            mask = plt.imread(f'{mask_dir}/{mask_file}')
            if mask.ndim == 3:
                mask = mask[:, :, 0]
            if mask.shape[:2] != img_size:
                mask = resize(mask, img_size, preserve_range=True,
anti_aliasing=False)
            mask = (mask > 0.5).astype(np.float32)

            images.append(img.astype(np.float32))
            masks.append(mask)

        X, y = np.array(images), np.array(masks)
        print(f"Размерность данных: X {X.shape}, y {y.shape}")

        split       =       block_split(positions['x'].to_numpy(),
positions['y'].to_numpy(),
                                        y.mean(axis=(1,             2)),
int(positions['size'].iloc[0]),
                                        block_size,          train_size,
val_size)
        if split is None:
```

```
            print("Не удалось разделить датасет: слишком мало
патчей")
            return None
        (train_idx, val_idx, test_idx), cell_set = split

        X_train, y_train = X[train_idx], y[train_idx]
        X_val, y_val = X[val_idx], y[val_idx]
        X_test, y_test = X[test_idx], y[test_idx]

        n_used = len(train_idx) + len(val_idx) + len(test_idx)
        n_cells = [list(cell_set.values()).count(k) for k in
range(3)]
        print(f"\nКлетки {block_size}x{block_size}: Train
{n_cells[0]}, Val {n_cells[1]}, Test {n_cells[2]}")
        print(f"Исключено патчей на границах клеток разных выборок:
{len(X) - n_used}")
        print(f"\nРазделение: Train {X_train.shape[0]}, Val
{X_val.shape[0]}, Test {X_test.shape[0]}")
        for name, part in [('Train', y_train), ('Val', y_val),
('Test', y_test)]:
            print(f"  {name:5}: {len(part) / n_used * 100:4.1f}%
патчей, вода {part.mean() * 100:.1f}% пикселей")

        dataset = {
            'X_train': X_train, 'y_train': y_train,
            'X_val': X_val, 'y_val': y_val,
            'X_test': X_test, 'y_test': y_test
        }

        split_info = {
            'positions': positions, 'block_size': block_size,
'cell_set': cell_set,
            'train_idx': train_idx, 'val_idx': val_idx,
'test_idx': test_idx
        }
```

```
        return (X_train, y_train), (X_val, y_val), (X_test, y_test)

    def get_augmentation(p=0.5):
        """
        Аугментация позволяет искусственно увеличить размер
        датасета за счет случайных преобразований изображений.
        """
        return A.Compose([
            A.RandomRotate90(p=0.5),
            A.HorizontalFlip(p=0.5),
            A.VerticalFlip(p=0.5),
            A.Transpose(p=0.3),
            A.Affine(translate_percent=(-0.1,  0.1),  scale=(0.9,
1.1), rotate=(-30, 30), p=0.3),

            A.RandomBrightnessContrast(brightness_limit=0.2,
contrast_limit=0.2, p=0.5),
            A.ColorJitter(brightness=0.2,           contrast=0.2,
saturation=0.2, hue=0.1, p=0.4),
            A.HueSaturationValue(hue_shift_limit=15,
sat_shift_limit=25, val_shift_limit=15, p=0.3),

            A.GaussNoise(std_range=(0.012, 0.025), p=0.3),
            A.ISONoise(color_shift=(0.01, 0.05), p=0.2),
            A.GaussianBlur(blur_limit=(3, 5), p=0.3),
            A.MedianBlur(blur_limit=5, p=0.2),

            A.RandomFog(fog_coef_range=(0.1, 0.3), p=0.2),
            A.RandomRain(slant_range=(-10,   10),   drop_width=1,
p=0.1),
            A.RandomSnow(snow_point_range=(0.1, 0.3), p=0.1),

            A.CoarseDropout(num_holes_range=(8,               8),
hole_height_range=(32, 32),
```

```
                                hole_width_range=(32,   32),   fill=0,
p=0.2)
        ], p=p)


    def augment_dataset(X, y, augmentor=None):
        """
        Явная аугментация обучающей выборки: каждый патч
пропускается через
        get_augmentation, которая с вероятностью p случайно
преобразует его.
        Размер выборки не меняется, часть патчей остается без
изменений
        """
        augmentor = augmentor or get_augmentation()
        images, masks = [], []
        for img, mask in zip(X, y):
            augmented       =       augmentor(image=(img       *
255).astype(np.uint8), mask=mask.astype(np.uint8))
            images.append(augmented['image'].astype(np.float32)  /
255.0)
            masks.append(augmented['mask'].astype(np.float32))
        return np.stack(images), np.stack(masks)


    class UNet:
        def __init__(self, train_dir='model_train'):
            self.train_dir = f'/kaggle/working/{train_dir}'
            os.makedirs(self.train_dir, exist_ok=True)

        def   conv_block(self,   x,   n_filters,   kernel_size=3,
activation='relu', padding='same', dropout=0.1):
            """
            Сверточный блок для U-Net
```

```
            Состоит из двух сверточных слоев с BatchNormalization
и
            активацией ReLU.
            """
            x     =     layers.Conv2D(n_filters,     kernel_size,
padding=padding)(x)
            x = layers.BatchNormalization()(x)
            x = layers.Activation(activation)(x)

            x     =     layers.Conv2D(n_filters,     kernel_size,
padding=padding)(x)
            x = layers.BatchNormalization()(x)
            x = layers.Activation(activation)(x)

            if dropout > 0:
                x = layers.Dropout(dropout)(x)

            return x


        def    build_unet(self,    input_shape=(256,    256,    3),
n_filters=64, n_classes=1):
            """
            Построение архитектуры U-Net
            U-Net состоит из двух частей:
            - Encoder (сжатие): уменьшает размер, увеличивает
            количество каналов
            -   Decoder   (восстановление):   увеличивает   размер,
используя
            skip-connections
            Skip-connections передают информацию от encoder к
decoder,
            что позволяет сохранить детали изображения
            """
            inputs = layers.Input(input_shape)
```

```
            c1 = self.conv_block(inputs, n_filters)
            p1 = layers.MaxPooling2D((2, 2))(c1)

            c2 = self.conv_block(p1, n_filters * 2)
            p2 = layers.MaxPooling2D((2, 2))(c2)

            c3 = self.conv_block(p2, n_filters * 4)
            p3 = layers.MaxPooling2D((2, 2))(c3)

            c4 = self.conv_block(p3, n_filters * 8)
            p4 = layers.MaxPooling2D((2, 2))(c4)

            c5 = self.conv_block(p4, n_filters * 16)

            u6 = layers.Conv2DTranspose(n_filters * 8, (2, 2),
strides=(2, 2), padding='same')(c5)
            u6 = layers.concatenate([u6, c4])
            c6 = self.conv_block(u6, n_filters * 8)

            u7 = layers.Conv2DTranspose(n_filters * 4, (2, 2),
strides=(2, 2), padding='same')(c6)
            u7 = layers.concatenate([u7, c3])
            c7 = self.conv_block(u7, n_filters * 4)

            u8 = layers.Conv2DTranspose(n_filters * 2, (2, 2),
strides=(2, 2), padding='same')(c7)
            u8 = layers.concatenate([u8, c2])
            c8 = self.conv_block(u8, n_filters * 2)

            u9 = layers.Conv2DTranspose(n_filters, (2, 2),
strides=(2, 2), padding='same')(c8)
            u9 = layers.concatenate([u9, c1])
            c9 = self.conv_block(u9, n_filters)
```

```
            outputs = layers.Conv2D(n_classes, (1, 1),
activation='sigmoid')(c9)

            return Model(inputs, outputs)

        def dice_loss(self, y_true, y_pred, smooth=1e-6):
            """
            DICE LOSS - функция потерь для сегментации
            Dice Loss = 1 - (2*|A∩B| + smooth) / (|A| + |B| +
smooth)
            где A - истинная маска, B - предсказанная маска
            Значение от 0 (идеально) до 1 (плохо)
            """
            y_true_f = tf.reshape(tf.cast(y_true, tf.float32), [-
1])
            y_pred_f = tf.reshape(tf.cast(y_pred, tf.float32), [-
1])
            intersection = tf.reduce_sum(y_true_f * y_pred_f)
            score = (2. * intersection + smooth) /
(tf.reduce_sum(y_true_f) + tf.reduce_sum(y_pred_f) + smooth)
            return 1. - score


        def tversky_loss(self, y_true, y_pred, alpha=0.7, beta=0.3,
smooth=1e-6):
            """
            TVERSKY LOSS - обобщение Dice Loss с разными весами для
ошибок
            Параметры:
            - alpha: вес ложноотрицательных ошибок (FN) -
пропущенная вода
            - beta: вес ложноположительных ошибок (FP) - ложная
тревога
            Для обнаружения разливов важно alpha > beta (пропуск
разлива хуже ложной тревоги)
```

```
            """
            y_true = tf.cast(y_true, tf.float32)
            y_pred = tf.cast(y_pred, tf.float32)

            tp = tf.reduce_sum(y_true * y_pred)
            fn = tf.reduce_sum(y_true * (1 - y_pred))
            fp = tf.reduce_sum((1 - y_true) * y_pred)

            tversky = (tp + smooth) / (tp + alpha * fn + beta * fp
+ smooth)
            return 1 - tversky


        def combined_loss(self, alpha=0.7, beta=0.3):
            """
            Комбинированная функция потерь
            BCE (Binary Cross-Entropy) дает стабильность,
            Tversky дает точность на границах объектов
            """
            def loss(y_true, y_pred):
                bce  =  tf.keras.losses.binary_crossentropy(y_true,
y_pred)
                tv = self.tversky_loss(y_true, y_pred, alpha, beta)
                return bce + tv
            return loss


        def demonstrate_unet(self):
            """
            Демонстрация создания U-Net и функций потерь
            """
            print("\n" + "=" * 60)
            print("Этап 4: Реализация U-Net")
            print("=" * 60)
```

```
            unet = self.build_unet(input_shape=(256, 256, 3),
n_filters=32)
            print(f"Модель создана! Параметров:
{unet.count_params():,}")
            unet.summary()

            y_true = tf.constant([[[1.0], [0.0]], [[0.0],
[1.0]]])
            y_pred = tf.constant([[[0.9], [0.1]], [[0.2],
[0.8]]])

            print(f"  Dice Loss: {self.dice_loss(y_true,
y_pred).numpy():.4f}")
            print(f"  Tversky Loss (α=0.7, β=0.3):
{self.tversky_loss(y_true, y_pred).numpy():.4f}")

            return unet


        def train_model(self, model, X_train, y_train, X_val,
y_val, loss_fn, model_name, epochs=5,
                        augment=True, batch_size=8):
            """
            Обучение модели с заданной функцией потерь
            Используются колбэки:
            - ModelCheckpoint: сохраняет лучшую модель
            - EarlyStopping: останавливает обучение при отсутствии
прогресса
            - ReduceLROnPlateau: уменьшает learning rate при застое
            При augment=True патчи обучающей выборки случайно
аугментируются
            (валидационная выборка не аугментируется)
            """
            print(f"\nОбучение {model_name}...")
```

```
            if X_train.max() > 1.0:
                X_train = X_train.astype('float32') /
X_train.max()
                X_val = X_val.astype('float32') / X_val.max()

            if augment:
                X_train, y_train = augment_dataset(X_train,
y_train)

            binary_iou =
tf.keras.metrics.BinaryIoU(target_class_ids=[1], threshold=0.5,
name='iou')

            model.compile(

optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
                loss=loss_fn,
                metrics=['accuracy', binary_iou]
            )

            callbacks = [
                ModelCheckpoint(
                    f'{self.train_dir}/{model_name}.keras',
                    save_best_only=True,
                    monitor='val_iou', mode='max',
                    verbose=1
                ),
                EarlyStopping(
                    patience=30, restore_best_weights=True,
                    monitor='val_iou', mode='max', verbose=1
                ),
                ReduceLROnPlateau(
                    factor=0.5, patience=5, min_lr=1e-6,
                    verbose=1
                )
```

```
            ]

            history = model.fit(
                X_train, y_train,
                validation_data=(X_val, y_val),
                batch_size=batch_size, epochs=epochs,
                shuffle=True, callbacks=callbacks, verbose=1
            )

            return history


        def experiment_tversky_params(self, X_train, y_train,
X_val, y_val, param_grid=None, epochs=20, sample_size=100):
            """
            Проверяет разные комбинации alpha и beta:
            - (0.3,0.7): приоритет FP (меньше ложных тревог)
            - (0.5,0.5): равновесный (как Dice)
            - (0.7,0.3): приоритет FN (меньше пропусков) - для
экологии
            - (0.8,0.2): сильный приоритет FN
            - (0.9,0.1): экстремальный приоритет FN
            """
            print("\n" + "=" * 60)
            print("Исследование параметров TVERSKY")
            print("=" * 60)

            import pandas as pd

            if len(X_train) > sample_size:
                idx = np.random.choice(len(X_train), sample_size,
replace=False)
                X_train_s, y_train_s = X_train[idx], y_train[idx]
            else:
                X_train_s, y_train_s = X_train, y_train
```

```
            if len(X_val) > sample_size // 2:
                idx = np.random.choice(len(X_val), sample_size //
2, replace=False)
                X_val_s, y_val_s = X_val[idx], y_val[idx]
            else:
                X_val_s, y_val_s = X_val, y_val

            param_grid = param_grid or [
                (0.3, 0.7), (0.5, 0.5), (0.7, 0.3), (0.8, 0.2),
(0.9, 0.1)
            ]

            results = []
            for alpha, beta in param_grid:
                print(f"\nТестирование α={alpha}, β={beta}...")

                model = self.build_unet(input_shape=(256, 256, 3),
n_filters=32)

                def tversky_loss_current(y_true, y_pred):
                    return    self.tversky_loss(y_true,    y_pred,
alpha, beta)


model.compile(optimizer=tf.keras.optimizers.Adam(3e-3),
loss=tversky_loss_current)
                model.fit(
                    X_train_s, y_train_s,
                    validation_data=(X_val_s, y_val_s),
                    batch_size=8, epochs=epochs, verbose=0
                )

                y_pred = model.predict(X_val_s, verbose=0)
                y_pred_bin = (y_pred > 0.5).astype(np.float32)
```

```
                ious, recalls, precisions = [], [], []
                for i in range(len(X_val_s)):
                    true_bin, pred_bin = y_val_s[i] > 0.5,
y_pred_bin[i, :, :, 0] > 0.5
                    tp = np.sum(true_bin & pred_bin)
                    fp = np.sum(~true_bin & pred_bin)
                    fn = np.sum(true_bin & ~pred_bin)

                    ious.append(tp / (tp + fp + fn + 1e-10))
                    recalls.append(tp / (tp + fn + 1e-10))
                    precisions.append(tp / (tp + fp + 1e-10))

                results.append({
                    'alpha': alpha, 'beta': beta,
                    'IoU': np.mean(ious), 'Recall':
np.mean(recalls),
                    'Precision': np.mean(precisions),
                    'F1': 2 * np.mean(precisions) *
np.mean(recalls) / (np.mean(precisions) + np.mean(recalls) + 1e-10)
                })

                print(f"  IoU: {results[-1]['IoU']:.4f}, F1:
{results[-1]['F1']:.4f}")

            results_df = pd.DataFrame(results)
            self.plot_tversky_experiment(results_df)


results_df.to_csv(f'{self.train_dir}/tversky_experiment.csv',
index=False)
            return results_df


        def plot_tversky_experiment(self, results_df):
```

```
            """
            Визуализация результатов эксперимента С TVERSKY LOSS
            Два графика:
            - Сравнение IoU, Recall, Precision для разных
параметров
            - F1-score для выбора лучших параметров
            """
            fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

            x = [f"α={r:.1f}\nβ={b:.1f}" for r, b in
zip(results_df['alpha'], results_df['beta'])]
            x_pos = np.arange(len(x))
            width = 0.25

            ax1.bar(x_pos - width, results_df['IoU'], width,
label='IoU', alpha=0.8, color='green')
            ax1.bar(x_pos, results_df['Recall'], width,
label='Recall', alpha=0.8, color='blue')
            ax1.bar(x_pos + width, results_df['Precision'], width,
label='Precision', alpha=0.8, color='red')
            ax1.set_xticks(x_pos)
            ax1.set_xticklabels(x)
            ax1.legend()
            ax1.grid(True, alpha=0.3)

            best_idx = results_df['F1'].argmax()
            ax2.bar(x_pos, results_df['F1'], width, alpha=0.8,
color='orange')
            ax2.set_xticks(x_pos)
            ax2.set_xticklabels(x)
            ax2.axhline(y=results_df['F1'].max(), color='red',
linestyle='--')
            ax2.grid(True, alpha=0.3)

            plt.tight_layout()
```

```
plt.savefig(f'{self.train_dir}/tversky_experiment.png',    dpi=150,
bbox_inches='tight')
            plt.show()

     (X_train,  y_train),  (X_val,  y_val),  (X_test,  y_test)  =
load_dataset(data_dir="/kaggle/input/datasets/shlyapa88man/water-
data/water_dataset")

     unet_web = UNet()
     model = unet_web.build_unet(n_filters=16)
     history = unet_web.train_model(
         model,
         X_train, y_train,
         X_val, y_val,
         loss_fn=unet_web.dice_loss,
         model_name='unet_dice',
         epochs=50
     )
     print(history.history)

     unet_web = UNet()
     model = unet_web.build_unet(n_filters=16)
     history = unet_web.train_model(
         model,
         X_train, y_train,
         X_val, y_val,
         loss_fn=unet_web.tversky_loss,
         model_name='unet_tversky',
         epochs=50
     )
     print(history.history)

     unet_web = UNet()
     model = unet_web.build_unet(n_filters=16)
```

```
    history = unet_web.train_model(
        model,
        X_train, y_train,
        X_val, y_val,
        loss_fn=unet_web.combined_loss(),
        model_name='unet_combined',
        epochs=50
    )
    print(history.history)

    unet_web = UNet()
    tversky_results = unet_web.experiment_tversky_params(
        X_train, y_train,
        X_val, y_val,
        epochs=20
    )
    tversky_results

    from skimage.morphology import remove_small_objects, closing,
disk
    from skimage.filters import threshold_otsu

    TRAIN_DIR    =    '/kaggle/input/datasets/shlyapa88man/water-
data/model_train'



    def postprocess_mask(mask, min_size=50):
        """
        Постобработка бинарной маски
        Удаляет маленькие объекты и заполняет дыры
        """
        mask_bool = mask > 0.5
        mask_clean        =        remove_small_objects(mask_bool,
min_size=min_size)
        return closing(mask_clean, disk(2)).astype(np.float32)
```

```
    def calculate_metrics(true_mask, pred_mask):
        """
        Расчет метрик качества сегментации
        IoU (Intersection over Union) - основная метрика
        """
        true_bin, pred_bin = true_mask > 0.5, pred_mask > 0.3
        tp = np.sum(true_bin & pred_bin)
        fp = np.sum(~true_bin & pred_bin)
        fn = np.sum(true_bin & ~pred_bin)
        return {'IoU': tp / (tp + fp + fn + 1e-10), 'TP': tp, 'FP':
fp, 'FN': fn}


    def find_optimal_threshold(model, X_val, y_val, model_name="",
thresholds=np.arange(0.3, 0.9, 0.05)):
        """
        Поиск оптимального порога бинаризации для модели
        Перебирает пороги от 0.3 до 0.85 и выбирает тот,
        при котором IoU на валидационной выборке максимален.
        """
        print(f"\nПоиск оптимального порога для {model_name}...")

        best_threshold, best_iou = 0.5, 0
        threshold_results = []

        n_samples = min(30, len(X_val))
        X_sample, y_sample = X_val[:n_samples], y_val[:n_samples]
        preds     =     model.predict(X_sample,     batch_size=4,
verbose=0)[..., 0]

        for threshold in thresholds:
            ious = []
            for i in range(n_samples):
```

```
                pred_bin = postprocess_mask((preds[i] > threshold).astype(np.float32))
                iou = calculate_metrics(y_sample[i], pred_bin)['IoU']
                ious.append(iou)

            mean_iou = np.mean(ious)
            threshold_results.append((threshold, mean_iou))

            if mean_iou > best_iou:
                best_iou, best_threshold = mean_iou, threshold
                print(f"Порог {threshold:.2f}: IoU = {mean_iou:.4f} (НОВЫЙ ЛУЧШИЙ)")
            else:
                print(f"Порог {threshold:.2f}: IoU = {mean_iou:.4f}")

        plot_threshold_analysis(threshold_results, model_name)

        return best_threshold, best_iou


    def plot_threshold_analysis(threshold_results, model_name):
        """
        ВИЗУАЛИЗАЦИЯ ЗАВИСИМОСТИ IoU ОТ ПОРОГА
        Показывает, как меняется качество сегментации при изменении
        порога, и отмечает оптимальное значение.
        """
        thresholds, ious = zip(*threshold_results)

        plt.figure(figsize=(10, 6))
        plt.plot(thresholds, ious, 'b-o', linewidth=2, markersize=8)
        plt.xlabel('Порог бинаризации')
        plt.ylabel('IoU')
```

```
        plt.title(f'Зависимость качества от порога\n{model_name}')
        plt.grid(True, alpha=0.3)

        best_idx = np.argmax(ious)
        best_threshold,    best_iou    =    thresholds[best_idx],
ious[best_idx]

        plt.axvline(x=best_threshold, color='red', linestyle='--',
alpha=0.7)
        plt.scatter([best_threshold],   [best_iou],   color='red',
s=200, zorder=5)

        plt.annotate(
            f'Оптимум: {best_threshold:.2f}\nIoU={best_iou:.4f}',
            xy=(best_threshold,   best_iou),   xytext=(10,   -35),
textcoords='offset points',
            fontsize=10, color='red'
        )

        plt.tight_layout()
        plt.savefig(

f'/kaggle/working/model_train/threshold_analysis_{model_name.repla
ce(" ", "_")}.png',
            dpi=150, bbox_inches='tight'
        )
        plt.show()



    def    visualize_predictions(models_dict,    X_test,    y_test,
n_samples=4, save=True):
        """
        Визуализация предсказаний моделей с адаптивными порогами
        Показывает для каждого тестового примера:
        - оригинал
```

```
        - идеальную маску
        - предсказания всех моделей
        - карту ошибок лучшей модели
        """
        print("\n" + "=" * 60)
        print("Визуализация предсказаний")
        print("=" * 60)

        indices = np.random.choice(len(X_test), min(n_samples,
len(X_test)), replace=False)
        n_models = len(models_dict)

        fig, axes = plt.subplots(len(indices), n_models + 3,
figsize=(4 * (n_models + 3), 4 * len(indices)))
        if len(indices) == 1:
            axes = axes.reshape(1, -1)

        optimal_thresholds = {'Dice Loss': 0.65, 'Tversky Loss':
0.5, 'Combined Loss': 0.45}

        for row, idx in enumerate(indices):
            img, true_mask = X_test[idx], y_test[idx].squeeze()
            col = 0

            axes[row, col].imshow(img)
            axes[row, col].set_title('Снимок')
            axes[row, col].axis('off')
            col += 1

            axes[row, col].imshow(true_mask, cmap='Blues', vmin=0,
vmax=1)
            axes[row, col].set_title('Идеальная маска')
            axes[row, col].axis('off')
            col += 1
```

```
            ious = []
            for name, model in models_dict.items():
                pred_raw = model.predict(img[np.newaxis],
verbose=0)[0, :, :, 0]
                threshold = optimal_thresholds.get(name, 0.5)

                if name == 'Dice Loss' and pred_raw.mean() > 0.7:
                    try:
                        pred_norm = ((pred_raw - pred_raw.min()) /
(pred_raw.max() - pred_raw.min() + 1e-8) * 255).astype(np.uint8)
                        threshold = threshold_otsu(pred_norm) /
255.0
                    except:
                        threshold = 0.7

                pred_bin = postprocess_mask((pred_raw >
threshold).astype(np.float32))
                iou = calculate_metrics(true_mask,
pred_bin)['IoU']
                ious.append((name, iou))

                axes[row, col].imshow(pred_bin, cmap='Blues',
vmin=0, vmax=1)
                axes[row,
col].set_title(f'{name}\nпорог={threshold:.2f}\nIoU={iou:.3f}',
fontsize=9)
                axes[row, col].axis('off')
                col += 1

            if ious:
                best_name, best_iou = max(ious, key=lambda x: x[1])
                best_model = models_dict[best_name]
                pred_raw = best_model.predict(img[np.newaxis],
verbose=0)[0, :, :, 0]
                threshold = optimal_thresholds.get(best_name, 0.5)
```

```
                pred_bin = postprocess_mask((pred_raw > threshold).astype(np.float32))

                error_map = np.zeros((*pred_bin.shape, 3))
                tp_mask = (true_mask > 0.5) & (pred_bin > 0.5)
                fp_mask = (true_mask <= 0.5) & (pred_bin > 0.5)
                fn_mask = (true_mask > 0.5) & (pred_bin <= 0.5)
                error_map[tp_mask] = [0, 1, 0]
                error_map[fp_mask] = [1, 0, 0]
                error_map[fn_mask] = [1, 1, 0]

                axes[row, -1].imshow(error_map)
                axes[row, -1].set_title(f'Ошибки\n{best_name}: {best_iou:.3f}', fontsize=10)
                axes[row, -1].axis('off')

        plt.tight_layout()
        if save:

plt.savefig(f'/kaggle/working/model_train/test_predictions.png', dpi=150, bbox_inches='tight')
        plt.show()


    def visualize_predictions_with_thresholds(models_dict, X_test, y_test, optimal_thresholds=None, n_samples=4, save=True):
        """
        Визуализация предсказаний с оптимальными порогами
        Использует найденные ранее оптимальные пороги для каждой модели.
        """
        print("\n" + "=" * 60)
        print("Визуализация - оптимальные пороги")
        print("=" * 60)
```

```
        if optimal_thresholds is None:
            optimal_thresholds = {name: 0.5 for name in
models_dict.keys()}

        indices = np.random.choice(len(X_test), min(n_samples,
len(X_test)), replace=False)
        n_models = len(models_dict)

        fig, axes = plt.subplots(len(indices), n_models + 3,
figsize=(4 * (n_models + 3), 4 * len(indices)))
        if len(indices) == 1:
            axes = axes.reshape(1, -1)

        for row, idx in enumerate(indices):
            img, true_mask = X_test[idx], y_test[idx].squeeze()
            col = 0

            axes[row, col].imshow(img)
            axes[row, col].set_title('Снимок')
            axes[row, col].axis('off')
            col += 1

            axes[row, col].imshow(true_mask, cmap='Blues', vmin=0,
vmax=1)
            axes[row, col].set_title('Идеальная маска')
            axes[row, col].axis('off')
            col += 1

            ious = []
            for name, model in models_dict.items():
                pred_raw = model.predict(img[np.newaxis],
verbose=0)[0, :, :, 0]
                threshold = optimal_thresholds.get(name, 0.5)
                pred_bin = postprocess_mask((pred_raw >
threshold).astype(np.float32))
```

```
                iou = calculate_metrics(true_mask,
pred_bin)['IoU']
                ious.append((name, iou))

                axes[row, col].imshow(pred_bin, cmap='Blues',
vmin=0, vmax=1)
                axes[row,
col].set_title(f'{name}\nпорог={threshold:.2f}\nIoU={iou:.3f}',
fontsize=9)
                axes[row, col].axis('off')
                col += 1

            if ious:
                best_name, best_iou = max(ious, key=lambda x: x[1])
                best_model = models_dict[best_name]
                pred_raw = best_model.predict(img[np.newaxis],
verbose=0)[0, :, :, 0]
                threshold = optimal_thresholds.get(best_name, 0.5)
                pred_bin = postprocess_mask((pred_raw >
threshold).astype(np.float32))

                error_map = np.zeros((*pred_bin.shape, 3))
                tp_mask = (true_mask > 0.5) & (pred_bin > 0.5)
                fp_mask = (true_mask <= 0.5) & (pred_bin > 0.5)
                fn_mask = (true_mask > 0.5) & (pred_bin <= 0.5)
                error_map[tp_mask] = [0, 1, 0]
                error_map[fp_mask] = [1, 0, 0]
                error_map[fn_mask] = [1, 1, 0]

                axes[row, -1].imshow(error_map)
                axes[row, -1].set_title(f'Ошибки\n{best_name}:
{best_iou:.3f}', fontsize=10)
                axes[row, -1].axis('off')

        plt.tight_layout()
```

```
        if save:

plt.savefig(f'/kaggle/working/model_train/test_predictions_optimal
.png', dpi=150, bbox_inches='tight')
        plt.show()


    models = {
        'Dice                                                    Loss':
tf.keras.models.load_model(f'{TRAIN_DIR}/unet_dice.keras',
compile=False),
        'Tversky                                                 Loss':
tf.keras.models.load_model(f'{TRAIN_DIR}/unet_tversky.keras',
compile=False),
        'Combined                                                Loss':
tf.keras.models.load_model(f'{TRAIN_DIR}/unet_combined.keras',
compile=False),
    }
    thresholds = {}
    for name, model in models.items():
        thresholds[name], _ = find_optimal_threshold(model, X_val,
y_val, model_name=name,

thresholds=np.arange(0.3, 0.85, 0.05))

    np.random.seed(42)
    visualize_predictions(models, X_test, y_test, n_samples=4)
    np.random.seed(42)
    visualize_predictions_with_thresholds(models, X_test, y_test,
thresholds, n_samples=4)
```